

1. Swim Times.

This is a program written using Donald Knuth’s literate programming paradigm (see more below). It fetches best short-course yard (SCY) and long-course meter (LCM) times for a roster of swimmers. The roster began with four family swimmers:

Stella Julianna Evans — 10 Under Girls
Kalea Rose Benavente — 13–14 Girls
Kenneth Ray Evans — 11–12 Boys
Keith Santiago Evans — 11–12 Boys

and now also covers Katie Ledecky (as age-group windows) and the full College Area Swim Team (CAST) roster listed in the **SWIMMERS** table of the main program. Each swimmer carries a unique *id* used to select her on the command line; the ids are enumerated in the software requirements specification (**swim-times-srs.tex**). Thirty-one events are reported for each swimmer. Short-course yard (SCY) events: 50, 100, 200, 500, 1000, and 1650 Freestyle; 50 and 100 Butterfly; 50 and 100 Backstroke; 50 and 100 Breaststroke; and 100 and 200 Individual Medley. Long-course meter (LCM) events: 50, 100, 200, 400, 800, and 1500 Freestyle; 50, 100, and 200 Butterfly; 50, 100, and 200 Backstroke; 50, 100, and 200 Breaststroke; and 200 and 400 Individual Medley. Data is fetched live from the USA Swimming data hub’s public REST services at times-api.usaswimming.org.

2. How it works. The data hub was formerly powered by a Sisense JAQL analytics API; it has since moved to first-party REST services, and this program follows. We make two kinds of requests:

0. A *sign-in*, once per run, which exchanges a stored user name and password for a subject and session id and then activates that session (see *Sign-in*).
1. A *member search* (**GetMembersForFilters**) to resolve the swimmer’s *memberId*, given a name string and a substring to match the returned full name. When a swimmer’s *memberId* is already known it is taken from the roster and this call is skipped in favour of the cheaper **GET /GetMember/<id>**.
2. A *best-times fetch* (**POST /BestTimes**) issued once per stroke-and-distance pair; each record delivers a stroke, distance, course, and formatted time — from which the event code (e.g. **100 FR SCY**) is reassembled — together with the swim date, meet name, and the motivational standard attained (**B**, **BB**, **A**, **AA**, **AAA**, **AAAA**, or an elite label such as **Nats** or **Trials**).

All HTTP communication is handled by **libcurl**. JSON responses are scanned with simple string operations rather than a full parse tree.

3. Authentication, and the 2026 access change. The data hub has never used a bearer token for public data. A caller identifies itself with three headers — `AppName: DataHub`, a client-minted `Device-Id`, and `Usas-Sub-Id`, which carries either a signed-in subject or the literal string `Anonymous`. A signed-in caller adds two more, `Usas-Session-Id` and `rate-key`, the latter an HMAC-SHA256 of the current ten-second epoch bucket keyed by the session id.

Through 2025 the times endpoints answered anonymous callers, and this program relied on that. They no longer do. As of this writing the service partitions its surface in three:

- *Open to anonymous callers:* the reference feeds (`SearchFilter/GetAllEvents`, `SearchFilter/GetLscs`, `TimesSearch/GetTimeStandard`...) and the single-member lookup `TimesSearch/GetMember/<memberId>`.
- *Closed with 403 Forbidden:* every search and every times query — `GetMembersForFilters`, `BestTimes`, `GetBestTimesForMember`, `GetAllTimesForFilters`, `GetTopTimesLeaderBoard`, `Ranking/Search`, `Records/SFSearch`.
- *Rejected with 401 Unauthorized:* anything sent without a recognised `Usas-Sub-Id` at all.

The front end agrees: `POST security/auth/GetDataHubSecurityInfoForIdp` with subject `Anonymous` returns a route table containing only `"/`, so the browser redirects an anonymous visitor to the login page before it ever issues a times query.

Consequently the program *signs in*. Every online run begins by reading a user name and password from a credentials file and walking the data hub's OpenID Connect flow to obtain a subject and a session id, which then authenticate every request; the whole sequence is described under *Sign-in* below. There is no anonymous path and no anonymous fallback — an unauthenticated search or times query is simply a slower route to 403, so the program does not attempt one. `-o offline` needs no credentials at all, and `-o diag` reports the sign-in result together with a reachability table for all five endpoint classes.

4. Literate programming. Donald Knuth introduced *literate programming* in 1984 as a way of writing software that is meant to be read by human beings first and executed by computers second. Rather than annotating code with comments, a literate program interweaves prose and code in a single source document. The prose explains the *why*—the motivation, the design decisions, the mathematical reasoning—while the code expresses the *how*. The two live together in one file (conventionally given the extension `.w`) and are separated only at build time by two companion tools: `ctangle` and `cweave`.

5. ctangle. `ctangle` is the *tangling* tool. It reads a `.w` source file and extracts the C code sections, assembling them in the order dictated by named chunk references rather than the order in which they appear in the document. The result is a plain `.c` file that a standard C compiler can process without any knowledge of literate programming. In this project, running

```
ctangle swim-times.w
```

produces `swim-times.c`, which is then compiled with `cc` and linked against `libcurl` to create the `swim-times` executable. The generated `.c` file should be treated as a build artefact: the `.w` file is the true source of record.

6. cweave. `cweave` is the *weaving* tool. It reads the same `.w` source and produces a `.tex` file formatted for `pdftex` using the `cwebmac` macro package. `cweave` pretty-prints all C code with bold keywords, italic identifiers, and cross-references, and numbers every named chunk so the reader can follow the program's logical structure independently of its physical layout. An index of identifiers and a table of contents are generated automatically. Running

```
cweave swim-times.w
```

produces `swim-times.tex`; running `pdftex` on that file yields the typeset documentation you are reading now.

7. Compilation. After tangling with `ctangle`:

```
gcc -O2 -o swim-times swim-times.c $(curl-config --libs)
```

8. Program structure. The top-level arrangement of the tangled output is:

- ⟨ Includes [10](#) ⟩
- ⟨ Global constants [11](#) ⟩
- ⟨ Option flags [18](#) ⟩
- ⟨ Type definitions [23](#) ⟩
- ⟨ HTTP utilities [27](#) ⟩
- ⟨ JSON scanner [53](#) ⟩
- ⟨ Sign-in machinery [57](#) ⟩
- ⟨ Database pipe [78](#) ⟩
- ⟨ Person lookup [95](#) ⟩
- ⟨ Times fetch [109](#) ⟩
- ⟨ Offline fetch [126](#) ⟩
- ⟨ Main function [132](#) ⟩

9. Includes and constants.

```

10. <Includes 10> ≡
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <unistd.h>
#include <time.h>
#include <curl/curl.h>
#include <libpq-fe.h>

```

This code is used in section 8.

11. `TIMES_API` is the base URL of the USA Swimming public times service that backs `data.usaswimming.org`. `EVENTS` lists all event codes we query: twelve short-course yard (SCY) events plus two additional SCY distance events (1000 FR and 1650 FR) and seventeen long-course meter (LCM) events, for a total of thirty-one.

The global constants are split into three sub-modules so that no single module exceeds twenty-four lines.

```

<Global constants 11> ≡
  <Numeric constants 12> <Data hub credentials 13> <Event table 15>

```

This code is used in section 8.

12. The numeric limits used throughout the program, the two HTTP timeouts (so a hung data-hub connection cannot wedge a run indefinitely), and the process exit codes. `RC_NOPERM` is returned when the data hub refused the request for want of credentials and `RC_UNAVAIL` when it failed for any other reason, so a wrapper script can tell “sign in” from “try again later”.

```

<Numeric constants 12> ≡
#define NUM_EVENTS 31
#define MAX_TIMES 200
#define HTTP_TIMEOUT_SECS 30 /* whole-request ceiling */
#define HTTP_CONNECT_SECS 10 /* TCP+TLS handshake ceiling */
#define RC_OK 0 /* success */
#define RC_ERROR 1 /* generic failure */
#define RC_USAGE 2 /* bad command line */
#define RC_UNAVAIL 69 /* data hub unreachable */
#define RC_NOPERM 77 /* data hub refused: sign in */

```

This code is used in section 11.

13. The base URLs of the times service. Note there is deliberately no hard-coded bearer token here. The USA Swimming data hub retired the Sisense JAQL API this program originally targeted (its embedded token now returns 401 `db_unauthorized`, and the current front-end no longer exposes any Sisense credential). The replacement REST services at `times-api.usaswimming.org` authenticate with four custom headers — *AppName: DataHub*, a client-generated *Device – Id*, *Usas – Sub – Id* (the signed-in subject, or the literal *Anonymous*), and, for a signed-in caller, *Usas – Session – Id* together with a computed *rate – key*. The *Device – Id* is minted at run time by *device_id*, and the credentials come from the environment; nothing is baked into the source. `SWIMS_API` is the service root and `TIMES_API` the `TimesSearch` controller beneath it; the reference feeds used by `-o diag` hang off the root rather than the controller. `DIAG_MEMBER` is a well-known public *memberId* (Katie Ledecy's) used only as a probe target by the diagnostics.

⟨Data hub credentials 13⟩ ≡

```
static const char SWIMS_API[] = "https://times-api.usaswimming.org/swims";
static const char TIMES_API[] = "https://times-api.usaswimming.org/swims/TimesSearch";
⟨Sign-in endpoints 14⟩
#define DIAG_MEMBER "6CD35348E5824C"
#define HTTP_USER_AGENT "swim-times/3.0_␣(+literate_␣CWEB_␣client;_␣libcurl)"
```

This code is used in section 11.

14. The three services the sign-in sequence walks through: the data hub's back-end-for-front-end, which owns the OpenID Connect dance; the identity provider, which owns the password form; and the security service, whose `SECURITY_INFO` endpoint both reports the caller's permissions and — crucially — activates the new session for the times API. `ENV_FILE_DEFAULT` is where the credentials live when `USAS_ENV_FILE` does not say otherwise.

⟨Sign-in endpoints 14⟩ ≡

```
static const char BFF_LOGIN[] = "https://dhy-prod.usaswimming.org/bff/login?returnUrl=%2F";
static const char BFF_SIGNIN[] = "https://dhy-prod.usaswimming.org/signin-oidc";
static const char BFF_USERINFO[] = "https://dhy-prod.usaswimming.org/bff/userinfo";
static const char IDP_LOGIN[] = "https://login.usaswimming.org/Login";
static const char SECURITY_INFO[] = "https://security-api.usaswimming.org/se\
curity/auth/" "GetDataHubSecurityInfoForIdp";
#define ENV_FILE_DEFAULT "/.usas-env"
```

This code is used in section 13.

15. The event-code table is split into SCY and LCM sub-lists so the array initialiser fits within the line limit.

⟨Event table 15⟩ ≡

```
static const char *EVENTS[NUM_EVENTS] = {⟨SCY events 16⟩⟨LCM events 17⟩};
```

This code is used in section 11.

16. The fourteen short-course yard event codes.

⟨SCY events 16⟩ ≡

```
"50_␣FR_␣SCY", "100_␣FR_␣SCY", "200_␣FR_␣SCY", "500_␣FR_␣SCY", "1000_␣FR_␣SCY", "1650_␣FR_␣SCY",
"50_␣FL_␣SCY", "100_␣FL_␣SCY", "50_␣BK_␣SCY", "100_␣BK_␣SCY", "50_␣BR_␣SCY", "100_␣BR_␣SCY",
"100_␣IM_␣SCY", "200_␣IM_␣SCY",
```

This code is used in section 15.

17. The seventeen long-course meter event codes.

`< LCM events 17 > ≡`

```
"50_FR_LCM", "100_FR_LCM", "200_FR_LCM", "400_FR_LCM", "800_FR_LCM", "1500_FR_LCM",
"50_FL_LCM", "100_FL_LCM", "200_FL_LCM", "50_BK_LCM", "100_BK_LCM", "200_BK_LCM",
"50_BR_LCM", "100_BR_LCM", "200_BR_LCM", "200_IM_LCM", "400_IM_LCM"
```

This code is used in section 15.

18. The program accepts two optional flags on the command line.

The `-o` flag takes a comma-separated list of one or more tokens. A token is either one of four *behaviour keywords* or a *swimmer id*:

- **fastest** — print only the single fastest time per event.
- **csv** — emit CSV lines (swimmer name on every line) instead of a table.
- **store** — write each row to the local Postgres database `swim-times` via `libpq`; combine with `csv` to also print to the terminal.
- **offline** — skip all network calls; read times from the local Postgres `swim-times` database instead.

Any other token is treated as a swimmer id (e.g. `stella`, `kalea`, `ledecky14`, `glass-layla`); see the **SWIMMERS** roster for the full list. When no swimmer id is given every swimmer is processed. Tokens may be combined, e.g. `-o stella,fastest` or `-o kalea,csv,store`. When no swimmer keyword is specified all swimmers (subject to mode) are shown.

The `-e` flag takes one or more comma-separated event codes (e.g. `-e "100 FR SCY,50 FL LCM"`) and restricts output to those events only. The flag may also be repeated (e.g. `-e "100 FR SCY" -e "50 FL LCM"`). When `-e` is omitted all thirty-one events are reported.

If no options are given at all the program prints a usage message and exits.

`< Option flags 18 > ≡`

```
#define OPT_FASTEST (1 << 0) /* print only the single fastest time */
#define OPT_CSV (1 << 1) /* emit CSV lines instead of a table */
#define OPT_STORE (1 << 2) /* also persist rows to Postgres */
#define OPT_OFFLINE (1 << 3) /* read from Postgres, skip network */
#define OPT_DIAG (1 << 4) /* probe data-hub reachability and exit */
#define OPT_SELFTEST (1 << 5) /* run the SHA-256/HMAC vectors and exit */
```

See also section 21.

This code is used in section 8.

19. `g_not_found` distinguishes the two ways a lookup can return `Λ`. “The service would not answer” must stop the run — the next swimmer will fare no better. “The service answered, and this person is not in it” must not: it is a fact about one roster entry, and the remaining swimmers are unaffected. Conflating them is a defect this revision repairs; see the person lookup. `g_missing` counts the second kind so the run can say at the end how many entries it skipped.

20. The three `g_cur_` variables carry the swimmer currently being processed from the person lookup, which is where the data hub reports her club and LSC, to `db_insert_row`, which needs them to fill in the shared schema’s *swimmer* row. They are globals rather than arguments because the path between the two runs through `fetch_times` and the shared sort-and-emit chunk, neither of which has any other reason to know about them.

21. These are the behaviour flags (how to render or where to read/write). Swimmer *selection*—which people to fetch—is no longer encoded as one bit per swimmer, because the roster now holds several dozen entries and would overflow the flag word. Instead each swimmer carries a unique string *id* (see the *Swimmer* record) and any `-o` token that is not one of the four behaviour keywords above is treated as a swimmer id and pushed onto *g_sel*. When *g_nsel* is zero every swimmer is processed; otherwise only those whose *id* appears in *g_sel*.

⟨ Option flags 18 ⟩ +=

```
#define MAX_SEL 64    /* max distinct swimmer ids selectable at once */
static int g_opts = 0; /* behaviour flags; see above */
static const char *g_sel[MAX_SEL]; /* selected swimmer ids (heap copies) */
static int g_nsel = 0; /* number of entries in g_sel */
static char g_events[NUM_EVENTS][32]; /* event codes requested via -e */
static int g_nevents = 0; /* number of entries in g_events */
static const char *g_member_id = Λ; /* ad-hoc memberId from -m */
static int g_auth_blocked = 0; /* set when the hub answered 401 or 403 */
static int g_not_found = 0; /* last lookup concluded "no such swimmer" */
static int g_missing = 0; /* swimmers skipped for that reason */
static const char *g_cur_member = Λ; /* memberId of the swimmer in hand */
static char g_cur_lsc[16] = ""; /* her LSC code, when the lookup gave one */
static char g_cur_club[128] = ""; /* her club name, likewise */
```

22. Data structures.

23. A *Buffer* holds a dynamically-grown heap string. `libcurl` appends each response chunk to it via the write callback.

```

⟨Type definitions 23⟩ ≡
    typedef struct {
        char *data;
        size_t size;
    } Buffer;

```

See also sections 24 and 25.

This code is used in section 8.

24. A *TimeRow* records one swim result. *sort_key* is the swim time in seconds (smaller is faster), computed by *time_to_seconds*; *time* is the formatted string (e.g. 1:02.45); *date* is the swim date formatted as YYYY-MM-DD; *standard* is the motivational standard attained (e.g. B, BB, A); and *meet* is the meet name. The `BestTimes` feed populates every field; only when a swim earned no motivational standard is *standard* left empty.

```

⟨Type definitions 23⟩ +≡
    typedef struct {
        double sort_key;
        char time[32];
        char date[16];
        char standard[48];
        char meet[256];
    } TimeRow;

```

25. A *Swimmer* record holds the per-swimmer search parameters: a query string sent to the person-search API, a lower-case substring used to identify the correct row, a flag bit used to filter output, and an optional age window expressed as ISO YYYY-MM-DD dates. A sixth field, *member_id*, short-circuits the whole search when the swimmer’s data-hub identifier is already known — which matters now that `GetMembersForFilters` is closed to anonymous callers while `GetMember/<id>` remains open. A Λ *member_id* means “look her up by name”. The typedef lives in this section (rather than next to the `SWIMMERS` array in main) so that *offline_fetch* and *lookup_member_id* can reference it without a forward declaration.

```

⟨Type definitions 23⟩ +≡
    typedef struct {
        const char *id;      /* unique selection id, e.g. "stella" */
        const char *search_query; /* Name string to search for */
        const char *match_substr; /* Lower-case substring to match */
        const char *date_min; /* NULL or "YYYY-MM-DD" inclusive lower */
        const char *date_max; /* NULL or "YYYY-MM-DD" inclusive upper */
        const char *member_id; /* known memberId, or NULL to search for */
    } Swimmer;

```


26. HTTP utilities.

27. The `libcurl` write callback appends each incoming chunk to a **Buffer**, growing the allocation as needed. It maintains a null terminator so the buffer can always be treated as a C string.

⟨HTTP utilities 27⟩ ≡

```
static size_t write_cb(void *ptr, size_t size, size_t nmemb, void *ud)
{
    Buffer *b = (Buffer *) ud;
    size_t n = size * nmemb;
    char *p = realloc(b->data, b->size + n + 1);
    if (!p) return 0;
    b->data = p;
    memcpy(b->data + b->size, ptr, n);
    b->size += n;
    b->data[b->size] = '\0';
    return n;
}
```

See also sections 28, 30, 31, 32, 33, 34, 35, 36, 37, 39, 40, 41, 44, 48, and 49.

This code is used in section 8.

28. `base64` encodes a null-terminated string into standard base64. It is used only to shape the *Device – Id* header, so a compact implementation that never overflows `out` is all that is required.

⟨HTTP utilities 27⟩ +≡

```
static const char B64[] = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456\
789+/";

static void base64(const char *in, char *out, size_t out_sz)
{
    size_t len = strlen(in), i = 0, o = 0;
    while (i + 3 ≤ len ∧ o + 4 < out_sz) {
        unsigned t = ((unsigned char) in[i] << 16) | ((unsigned char) in[i + 1] << 8) | (unsigned
            char) in[i + 2];
        out[o++] = B64[(t >> 18) & 63];
        out[o++] = B64[(t >> 12) & 63];
        out[o++] = B64[(t >> 6) & 63];
        out[o++] = B64[t & 63];
        i += 3;
    }
    if (len > i ∧ o + 4 < out_sz) {
        size_t rem = len - i;
        unsigned t = (unsigned char) in[i] << 16;
        if (rem ≡ 2) t |= (unsigned char) in[i + 1] << 8;
        out[o++] = B64[(t >> 18) & 63];
        out[o++] = B64[(t >> 12) & 63];
        out[o++] = rem ≡ 2 ? B64[(t >> 6) & 63] : '=';
        out[o++] = '=';
    }
    out[o] = '\0';
}
```

29. SHA-256 and HMAC. A signed-in caller must present a **rate-key** header, which the data-hub front end computes as

$$\text{floor}(t/s) \text{ " " HMAC-SHA256}_{\text{sid}}(t)$$

where t is the current Unix time in milliseconds divided by 10^4 (a ten-second bucket) and s is the number of seconds elapsed since midnight UTC. Rather than take on a TLS-library dependency for one hash, the two primitives are implemented here in about sixty lines of portable C. FIPS 180-4 defines SHA-256 and RFC 2104 defines HMAC; the test suite checks both against the published vectors.

30. The SHA-256 state: eight chaining words, the total message length in bytes, and a 64-byte block accumulator.

```
< HTTP utilities 27 > +=
typedef struct {
    uint32_th[8];    /* chaining variables */
    uint64_tlen;     /* total bytes absorbed */
    unsigned char buf[64]; /* partial block */
    size_t n;        /* bytes currently held in buf */
} Sha256;
```

31. The sixty-four round constants: the first thirty-two bits of the fractional parts of the cube roots of the first sixty-four primes.

```
< HTTP utilities 27 > +=
static const uint32_tK256[64] = {#428a2f98U, #71374491U, #b5c0fbcfU, #e9b5dba5U, #3956c25bU,
    #59f111f1U, #923f82a4U, #ab1c5ed5U, #d807aa98U, #12835b01U, #243185beU, #550c7dc3U,
    #72be5d74U, #80deb1feU, #9bdc06a7U, #c19bf174U, #e49b69c1U, #efbe4786U, #0fc19dc6U,
    #240ca1ccU, #2de92c6fU, #4a7484aaU, #5cb0a9dcU, #76f988daU, #983e5152U, #a831c66dU,
    #b00327c8U, #bf597fc7U, #c6e00bf3U, #d5a79147U, #06ca6351U, #14292967U, #27b70a85U,
    #2e1b2138U, #4d2c6dfcU, #53380d13U, #650a7354U, #766a0abbU, #81c2c92eU, #92722c85U,
    #a2bfe8a1U, #a81a664bU, #c24b8b70U, #c76c51a3U, #d192e819U, #d6990624U, #f40e3585U,
    #106aa070U, #19a4c116U, #1e376c08U, #2748774cU, #34b0bcb5U, #391c0cb3U, #4ed8aa4aU,
    #5b9cca4fU, #682e6ff3U, #748f82eeU, #78a5636fU, #84c87814U, #8cc70208U, #90befffaU,
    #a4506cebU, #bef9a3f7U, #c67178f2U};
```

32. The four logical functions of the compression loop, written out as inline helpers so no preprocessor macros leak into the woven listing. *ror* is a 32-bit right rotation.

```

⟨ HTTP utilities 27 ⟩ +≡
    static uint32_t ror(uint32_t x, int n)
    {
        return (x >> n) | (x << (32 - n));
    }
    static uint32_t ep0(uint32_t x)
    {
        return ror(x, 2) ⊕ ror(x, 13) ⊕ ror(x, 22);
    }
    static uint32_t ep1(uint32_t x)
    {
        return ror(x, 6) ⊕ ror(x, 11) ⊕ ror(x, 25);
    }
    static uint32_t sg0(uint32_t x)
    {
        return ror(x, 7) ⊕ ror(x, 18) ⊕ (x >> 3);
    }
    static uint32_t sg1(uint32_t x)
    {
        return ror(x, 17) ⊕ ror(x, 19) ⊕ (x >> 10);
    }

```

33. *sha256_block* compresses one 64-byte block into the chaining state: the message schedule is expanded to sixty-four words and the eight working variables are stirred through sixty-four rounds.

(HTTP utilities 27) +≡

```
static void sha256_block(Sha256 *s, const unsigned char *p)
{
    uint32_t w[64], a, b, c, d, e, f, g, h, t1, t2;
    for (int i = 0; i < 16; i++)
        w[i] = (uint32_t)p[i*4] << 24 | (uint32_t)p[i*4+1] << 16 | (uint32_t)p[i*4+2] << 8 | (uint32_t)p[i*4+3];
    for (int i = 16; i < 64; i++) w[i] = w[i - 16] + sg0(w[i - 15]) + w[i - 7] + sg1(w[i - 2]);
    a = s->h[0];
    b = s->h[1];
    c = s->h[2];
    d = s->h[3];
    e = s->h[4];
    f = s->h[5];
    g = s->h[6];
    h = s->h[7];
    for (int i = 0; i < 64; i++) {
        t1 = h + ep1(e) + ((e & f) ⊕ (¬e & g)) + K256[i] + w[i];
        t2 = ep0(a) + ((a & b) ⊕ (a & c) ⊕ (b & c));
        h = g;
        g = f;
        f = e;
        e = d + t1;
        d = c;
        c = b;
        b = a;
        a = t1 + t2;
    }
    s->h[0] += a;
    s->h[1] += b;
    s->h[2] += c;
    s->h[3] += d;
    s->h[4] += e;
    s->h[5] += f;
    s->h[6] += g;
    s->h[7] += h;
}
```

34. *sha256_init* loads the standard initialisation vector — the first thirty-two bits of the fractional parts of the square roots of the first eight primes.

(HTTP utilities 27) +≡

```
static void sha256_init(Sha256 *s)
{
    static const uint32_t IV[8] = {#6a09e667U, #bb67ae85U, #3c6ef372U, #a54ff53aU, #510e527fU,
        #9b05688cU, #1f83d9abU, #5be0cd19U};
    memcpy(s->h, IV, sizeof IV);
    s->len = 0;
    s->n = 0;
}
```

35. *sha256_update* absorbs n bytes, compressing whenever the 64-byte accumulator fills. Byte-at-a-time absorption is ample here: the longest message the program hashes is a ten-digit bucket number.

(HTTP utilities 27) +≡

```
static void sha256_update(Sha256 *s, const void *data, size_t n)
{
    const unsigned char *p = (const unsigned char *) data;
    s-len += n;
    while (n-- > 0) {
        s-buf[s-n++] = *p++;
        if (s-n == 64) {
            sha256_block(s, s-buf);
            s-n = 0;
        }
    }
}
```

36. *sha256_final* applies the FIPS 180-4 padding — a 0x80 byte, zeros up to offset 56, then the 64-bit big-endian bit count — and serialises the chaining state. The bit count is captured *before* padding, since padding itself goes through *sha256_update* and so advances *s-len*.

(HTTP utilities 27) +≡

```
static void sha256_final(Sha256 *s, unsigned char out[32])
{
    uint64_t bits = s-len * 8;
    sha256_update(s, "\x80", 1);
    while (s-n != 56) sha256_update(s, "", 1);
    for (int i = 7; i >= 0; i--) s-buf[s-n++] = (unsigned char)(bits >> (i * 8));
    sha256_block(s, s-buf);
    s-n = 0;
    for (int i = 0; i < 8; i++) {
        out[i * 4] = (unsigned char)(s-h[i] >> 24);
        out[i * 4 + 1] = (unsigned char)(s-h[i] >> 16);
        out[i * 4 + 2] = (unsigned char)(s-h[i] >> 8);
        out[i * 4 + 3] = (unsigned char)(s-h[i]);
    }
}
```

37. *hmac_sha256* is RFC 2104 verbatim: a key longer than the block size is hashed down to thirty-two bytes, then padded to sixty-four; the inner digest of $\text{key} \oplus 0\text{x}36$ concatenated with the message is re-hashed under $\text{key} \oplus 0\text{x}5c$. The result is written to *hex* as sixty-four lower-case hexadecimal digits plus a terminator, so *hex* must hold at least sixty-five bytes.

⟨HTTP utilities 27⟩ +≡

```
static void hmac_sha256(const char *key, const char *msg, char *hex)
{
    unsigned char k[64], ip[64], op[64], ih[32], oh[32];
    size_t kl = strlen(key);
    Sha256 s;
    memset(k, 0, sizeof k);
    if (kl > 64) {
        sha256_init(&s);
        sha256_update(&s, key, kl);
        sha256_final(&s, k);
    }
    else memcpy(k, key, kl);
    for (int i = 0; i < 64; i++) {
        ip[i] = k[i]  $\oplus$  #36;
        op[i] = k[i]  $\oplus$  #5c;
    }
    sha256_init(&s);
    sha256_update(&s, ip, 64);
    sha256_update(&s, msg, strlen(msg));
    sha256_final(&s, ih);
    sha256_init(&s);
    sha256_update(&s, op, 64);
    sha256_update(&s, ih, 32);
    sha256_final(&s, oh);
    for (int i = 0; i < 32; i++) snprintf(hex + i * 2, 3, "%02x", oh[i]);
}
```

38. Data-hub credentials.

39. *usas_sub_id* and *usas_session_id* read the caller's identity from the environment. With `USAS_SUB_ID` unset the program presents itself as **Anonymous**, exactly as the front end does for a visitor who has not signed in — which is enough for the reference feeds and for **GetMember**, and is refused with **403** by everything else.

⌊ HTTP utilities 27 ⌋ +≡

```
static const char *usas_sub_id(void)
{
    const char *e = getenv("USAS_SUB_ID");
    return (e & *e) ? e : "Anonymous";
}

static const char *usas_session_id(void)
{
    const char *e = getenv("USAS_SESSION_ID");
    return (e & *e) ? e : Λ;
}
```

40. *build_rate_key* reproduces the front end's throttling token: the ten-second epoch bucket *t*, divided by the number of seconds elapsed since midnight UTC, then a dot, then the HMAC-SHA256 of the decimal spelling of *t* under the session id. *gmtime_r* supplies the UTC wall clock; a zero seconds-since-midnight (the first second of the day) is forced to one to avoid a division by zero, matching the front end's own zero guard.

⌊ HTTP utilities 27 ⌋ +≡

```
static void build_rate_key(const char *sid, char *out, size_t n)
{
    time_t now = time(Λ);
    long long t = (long long) now * 1000 / 10000;
    struct tm g;
    gmtime_r(&now, &g);
    long long secs = g.tm_hour * 3600 + g.tm_min * 60 + g.tm_sec;
    if (secs ≡ 0) secs = 1;
    char tbuf[32], hex[65];
    snprintf(tbuf, sizeof tbuf, "%lld", t);
    hmac_sha256(sid, tbuf, hex);
    snprintf(out, n, "%lld.%s", t / secs, hex);
}
```

41. *device_id* returns the cached *Device – Id* value the times service requires, building it once on first use. We reproduce the shape the data hub’s front-end produces: `base64 of "platform - vendor - fingerprint - millis"`, then the first five characters repeated after the fifteenth. The server validates this shape and answers 400 `Invalid Device-Id format in request headers` when it does not hold, so the duplication is load-bearing, not decoration. A per-run fingerprint (process id plus clock) satisfies the rest; `USAS_DEVICE_ID` overrides the whole computation for a caller who wants to pin one value across runs.

(HTTP utilities 27) +≡

```
static const char *device_id(void)
{
    static char id[512];
    const char *env = getenv("USAS_DEVICE_ID");
    if (id[0]) return id;
    if (env & *env) {
        snprintf(id, sizeof id, "%s", env);
        return id;
    }
    char raw[128], n[256];
    long now = (long) time(Λ);
    snprintf(raw, sizeof raw, "Linux_ _curlswim_ _%lx%lx_ _%ld000", (unsigned
        long) getpid(), (unsigned long) now, now);
    base64(raw, n, sizeof n);
    snprintf(id, sizeof id, "%.15s%.5s%s", n, n, n + 15);
    return id;
}
```


42. HTTP transport.

43. The defect this replaces. The original *http_request* returned *buf.data* directly and reported failure by returning Λ . That conflated two entirely different outcomes, because *buf.data* is left Λ whenever the write callback never fires — which is exactly what happens on *any* response with an empty body. The data hub answers a refused request with **403 Forbidden** and **content-length: 0**, so a policy rejection was indistinguishable from a DNS or TLS failure, and the caller printed "**person lookup request failed**" either way. The status code, the one piece of information that would have explained the problem, was never read at all.

The rewrite below fixes that. The HTTP status is retrieved with *curl_easy_getinfo* and returned through **status*; a successful transport that produced no body yields an empty string rather than Λ ; and Λ now means one thing only — the request never completed. The rewrite also sets connect and total timeouts (an unreachable host used to be able to hang a run indefinitely), follows redirects, negotiates compression, and sends a truthful *User-Agent*.

44. *http_request* performs one request — a GET when *body* is Λ , otherwise a POST carrying *body*. It returns the response body as a null-terminated heap string (possibly empty) and stores the HTTP status in **status*, or returns Λ when the request could not be completed at all, in which case **status* is zero. The caller frees the result.

$\langle$ HTTP utilities 27 $\rangle + \equiv$

```
static char *http_request(const char *url, const char *body, long *status) {  
     $\langle$  Initialize curl handle 45  $\rangle$   $\langle$  Issue HTTP request and return 47  $\rangle$ 
```

45. The easy handle is created, the data-hub headers are assembled, and all curl options are configured before the request is issued.

$\langle$ Initialize curl handle 45 $\rangle \equiv$

```
CURL *curl = curl_easy_init();  
if (status) *status = 0;  
if (!curl) return  $\Lambda$ ;  
Buffer buf = { $\Lambda$ , 0};  
struct curl_slist *hdrs =  $\Lambda$ ;  
 $\langle$  Build request headers 46  $\rangle$  curl_easy_setopt(curl, CURLOPT_URL, url);  
curl_easy_setopt(curl, CURLOPT_HTTPHEADER, hdrs);  
if (body) curl_easy_setopt(curl, CURLOPT_POSTFIELDS, body);  
curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, write_cb);  
curl_easy_setopt(curl, CURLOPT_WRITEDATA, &buf);  
curl_easy_setopt(curl, CURLOPT_USERAGENT, HTTP_USER_AGENT);  
curl_easy_setopt(curl, CURLOPT_FOLLOWLOCATION, 1L);  
curl_easy_setopt(curl, CURLOPT_ACCEPT_ENCODING, "");  
curl_easy_setopt(curl, CURLOPT_TIMEOUT, (long) HTTP_TIMEOUT_SECS);  
curl_easy_setopt(curl, CURLOPT_CONNECTTIMEOUT, (long) HTTP_CONNECT_SECS);
```

This code is used in section 44.

46. Every request carries `AppName`, `Device-Id`, and `Usas-Sub-Id`; the `Origin` and `Referer` pair identifies the data hub front end, which the edge WAF expects. A caller who exported `USAS_SESSION_ID` additionally gets `Usas-Session-Id` and a freshly computed `rate-key`. The header buffers are declared in this scope because `libcurl` does not copy the strings appended to an `slist` until the transfer runs.

There is deliberately no `Accept` header. The service's content negotiation treats `Accept: application/json` as a request to serialise its already-serialised payload a second time, returning the whole document as one JSON *string* with every inner quote escaped — which the scanner, looking for `"fullName"` and finding `\\"fullName\\"`, cannot read. Omitting `Accept` yields the plain object. The front end omits it too; the safe rule when talking to this service is to send exactly the headers its own client sends, and nothing more.

(Build request headers 46) $\equiv$

```
char dev_hdr[600], sub_hdr[160], sid_hdr[160], rk_hdr[224];
snprintf(dev_hdr, sizeof dev_hdr, "Device-Id:_%s", device_id());
snprintf(sub_hdr, sizeof sub_hdr, "Usas-Sub-Id:_%s", usas_sub_id());
hdrs = curl_slist_append(hdrs, "Content-Type:_application/json");
hdrs = curl_slist_append(hdrs, "AppName:_DataHub");
hdrs = curl_slist_append(hdrs, "Origin:_https://data.usaswimming.org");
hdrs = curl_slist_append(hdrs, "Referer:_https://data.usaswimming.org/");
hdrs = curl_slist_append(hdrs, sub_hdr);
hdrs = curl_slist_append(hdrs, dev_hdr);
const char *sid = usas_session_id();
if (sid) {
    char rk[128];
    build_rate_key(sid, rk, sizeof rk);
    snprintf(sid_hdr, sizeof sid_hdr, "Usas-Session-Id:_%s", sid);
    snprintf(rk_hdr, sizeof rk_hdr, "rate-key:_%s", rk);
    hdrs = curl_slist_append(hdrs, sid_hdr);
    hdrs = curl_slist_append(hdrs, rk_hdr);
}
```

This code is used in section 45.

47. The request is executed. A transport failure is reported here, once, with `libcurl`'s own description of it; an HTTP-level failure is left for the caller to interpret, since only the caller knows what it was asking for. An empty body is normalised to `" "` so that Λ unambiguously means “no response at all”.

(Issue HTTP request and return 47) $\equiv$

```
CURLcode rc = curl_easy_perform(curl);
long code = 0;
curl_easy_getinfo(curl, CURLINFO_RESPONSE_CODE, &code);
curl_slist_free_all(hdrs);
curl_easy_cleanup(curl);
if (rc != CURLE_OK) {
    fprintf(stderr, "Error: _cannot_reach_%s:_%s\n", url, curl_easy_strerror(rc));
    free(buf.data);
    return  $\Lambda$ ;
}
if (status) *status = code;
if (!buf.data) buf.data = calloc(1, 1);
return buf.data;
```

This code is used in section 44.

48. *http_status_text* names the handful of status codes this service actually returns, so a diagnostic reads HTTP 403 Forbidden rather than a bare number.

⟨HTTP utilities 27⟩ +≡

```
static const char *http_status_text(long s)
{
    switch (s) {
        case 200: return "OK";
        case 204: return "No_Content";
        case 400: return "Bad_Request";
        case 401: return "Unauthorized";
        case 403: return "Forbidden";
        case 404: return "Not_Found";
        case 429: return "Too_Many_Requests";
        case 500: return "Internal_Server_Error";
        case 502: return "Bad_Gateway";
        case 503: return "Service_Unavailable";
        default: return "";
    }
}
```

49. *report_http_error* prints one line naming the operation, the status, and the URL. The first 401 or 403 of a run additionally draws the sign-in advice; repeating it for each of the thirty-one events of each of twenty-nine swimmers would bury the information it is trying to convey. *g_auth_blocked* records that the run was refused for want of credentials so *main* can exit with RC_NOPERM rather than a generic failure.

⟨HTTP utilities 27⟩ +≡

```
static void report_http_error(const char *what, const char *url, long status)
{
    static int advised = 0;
    fprintf(stderr, "Error: %s failed --- HTTP %ld %s\n", what, status,
        http_status_text(status), url);
    if (status != 401 & status != 403) return;
    g_auth_blocked = 1;
    if (advised) return;
    advised = 1;
    ⟨Print authentication advice 50⟩
}
```

50. The advice is written once per run, to standard error, and names both ways forward: supply credentials, or fall back to the local mirror.

⟨Print authentication advice 50⟩ ≡

```
fputs("\n" "The USA Swimming Data Hub declined this request. The program\n"
    "did sign in, so the likely causes are\n"
    ", in order: \n" "1. The password in the credentials file is stale --- the\n"
    "sign-in step would have reported that, so check above\n"
    ".\n" "2. The session was not activated. Every run activates\n"
    "it via\n" "the security service; a 401 here means that call did\n"
    "not\n" "take effect. Re-run; if it persists, run -o diag.\n"
    "3. This account lacks permission for the endpoint (403)\n"
    ".\n" "Meanwhile -o offline reads times already mirrored in the\n"
    "local\n" "Postgres swim-times database, with no network at all.\n" "\n", stderr);
```

This code is used in section 49.

51. JSON scanner.

52. The REST responses are JSON arrays of objects with named fields. A member-search row looks like

```
"memberId":"6CD35348E5824C","fullName":"Katie Genevieve Ledecky",...
```

and a best-times row like

```
"strokeAbbreviation":"FR","distance":50,"courseCode":"SCY","swimTime":"22.64r"
```

Rather than build a full parse tree we scan forward for a named key and extract the immediately following value, advancing a position pointer. The scanner is format-agnostic: it served the old Sisense "text"/"data" cells and serves these named fields unchanged.

53. *scan_string* finds the first "key" at or after **pos*, copies the string value that follows into *out* (at most *max_len* - 1 bytes), and advances **pos* past it. Returns 1 on success, 0 if the key is absent.

(JSON scanner 53) $\equiv$

```
static int scan_string(const char *key, const char **pos, char *out, size_t max_len)
{
    char needle[128];
    snprintf(needle, sizeof needle, "\"%s\"", key);
    const char *p = strstr(*pos, needle);
    if (!p) return 0;
    p += strlen(needle);
    while (*p == ' ' || *p == ':') p++;
    if (*p == '"') return 0;
    p++;
    const char *e = strchr(p, '"');
    if (!e) return 0;
    size_t len = (size_t)(e - p);
    if (len >= max_len) len = max_len - 1;
    memcpy(out, p, len);
    out[len] = '\0';
    *pos = e + 1;
    return 1;
}
```

This code is used in section 8.

54. Numeric fields in these feeds (*distance*, *ages*, *ids*) are not needed by the scanner: every value the program consumes — meet name, event code, date, time, and standard — arrives as a JSON string, so a single string extractor suffices.

55. Sign-in.

The program signs in to the USA Swimming data hub before it fetches anything. This is not an option: since the hub withdrew anonymous access to its search and times endpoints there is no other way to retrieve a swim time, and a program that quietly fell back to an anonymous request would only be choosing a slower route to the same 403.

Credentials come from a file — `$USAS_ENV_FILE`, or `$HOME/.usas-env` — holding shell-style assignments:

```
export USAS_USER="..."
export USAS_PASS="..."
export SWIM_TIMES_PGCONNINFO="..."
```

The file is read, never executed, and *only names beginning USAS_ are honoured*. Two reasons. The narrow one is hygiene: a credentials file should not be able to reach into the rest of the environment. The pointed one is that such a file may well carry settings meant for *other* programs — the one this was written against also defines `SWIM_TIMES_PGCONNINFO`, pointing at a differently-shaped database — and silently adopting them would retarget this program's mirror without anyone asking for it. A deliberate override still works, because a value already present in the environment always wins over the file.

56. The sequence. Signing in takes six requests and one non-obvious seventh step.

1. GET `bff/login` — the data hub's back-end-for-front-end starts an OpenID Connect authorization-code flow with PKCE and redirects, eventually, to the identity provider's password form. We follow the redirects and keep the page.
2. POST `login.usaswimming.org/Login` — the form is resubmitted with the username, the password, the `ReturnUrl` it carried, and its antiforgery token. A successful sign-in answers 302; a failed one answers 200 and re-renders the form, so the status alone tells us which happened.
3. GET the authorization callback named by that redirect. Because the client registered `response_mode=form_post`, the reply is not another redirect but an HTML page whose body is a form that a browser would submit by script.
4. POST that form's four fields — `code`, `state`, `session_state`, `iss` — to `bff/signin-oidc`, which exchanges the code and sets the session cookie.
5. GET `bff/userinfo` — returns the claims, of which we need two: `sub`, the subject, and `sid`, the session id.
6. POST `security/auth/GetDataHubSecurityInfoForIdp` with those two.

Step 6 looks like a permissions query, and it is one — it returns the caller's name and the list of routes they may use. But it is also what *activates* the session for the times API. A session that has completed steps 1–5 and skipped step 6 is answered 401 `Unauthorized` by every protected times endpoint; the same session, after step 6, is answered 200. This was established by running the two orders against a fresh session repeatedly. The data hub's own front end issues the call on start-up, so the coupling is invisible from the browser — and it is the single most surprising thing in this program.

57. The sign-in machinery is assembled from six chunks.

```
<Sign-in machinery 57> ≡
  <Credentials file loader 58><HTML field scanner 60><Sign-in transport 62><Perform the OIDC
    sign-in 66><Activate the session 73><Ensure a signed-in session 74>
```

This code is used in section 8.

58. *load_env_file* reads the credentials file a line at a time. A line may begin with **export**; the value may be bare, single-quoted, or double-quoted; anything after a **#** at the start of a line is a comment. Nothing is executed and no shell is spawned.

⟨ Credentials file loader 58 ⟩ ≡

```
static void load_env_file(void) { char path[512];
    const char *p = getenv("USAS_ENV_FILE");
    if (p & *p) snprintf(path, sizeof path, "%s", p);
    else {
        const char *h = getenv("HOME");
        if (!h) return;
        snprintf(path, sizeof path, "%s%s", h, ENV_FILE_DEFAULT);
    }
    FILE *f = fopen(path, "r");
    if (!f) return;
    char
    line [2048] ;
    while ( fgets ( line , sizeof line , f ) ) ⟨ Apply one assignment 59 ⟩ fclose(f); }
```

This code is used in section 57.

59. One line is split at the first =, the value is unquoted in place, and the pair is published with *setenv*'s no-overwrite flag so that an explicit environment setting always wins. Only the **USAS_** prefix is honoured.

⟨ Apply one assignment 59 ⟩ ≡

```
{ char *s = line ;
    while (*s == ' ' || *s == '\t') s++;
    if (*s == '#' || *s == '\n' || !*s) continue;
    if (strncmp(s, "export", 7) == 0) s += 7;
    char *eq = strchr(s, '=');
    if (!eq) continue;
    *eq = '\0';
    char *val = eq + 1;
    size_t vl = strlen(val);
    while (vl & (val[vl - 1] == '\n' || val[vl - 1] == '\r')) val[--vl] = '\0';
    if (vl >= 2 & (*val == '"' || *val == '\'' || val[vl - 1] == *val)) {
        val[vl - 1] = '\0';
        val++;
    }
    if (strncmp(s, "USAS_", 5) == 0) setenv(s, val, 0);
}
```

This code is used in section 58.

60. *html_unescape* resolves, in place, the handful of entities the two forms actually contain. A `ReturnUrl` arrives with its query separators written `&`, and leaving them that way produces a `ReturnUrl` the identity provider rejects.

⟨HTML field scanner 60⟩ ≡

```
static void html_unescape(char *s)
{
    static const char *ENT[] = {"&amp;", "&lt;", "&gt;", "&quot;", "&#x27;", "&#39;", "&#x2F;",
                                "&#47;"};
    static const char REP[] = {'&', '<', '>', '"', '\\', '\'', '/', ' '};
    char *r = s, *w = s;
    while (*r) {
        size_t i = 0, n = sizeof REP;
        for (; i < n; i++) {
            size_t el = strlen(ENT[i]);
            if (strncmp(r, ENT[i], el) == 0) {
                *w++ = REP[i];
                r += el;
                break;
            }
        }
        if (i == n) *w++ = *r++;
    }
    *w = '\\0';
}
```

See also section 61.

This code is used in section 57.

61. *scan_field* finds the hidden input named *field* and copies its **value** attribute. The two pages quote their attributes differently — the identity provider’s form uses double quotes, the authorization callback’s uses single — so both spellings of the name are tried and whichever quote character opens the value closes it.

⟨HTML field scanner 60⟩ +≡

```
static int scan_field(const char *hay, const char *field, char *out, size_t max_len)
{
    char n1[96], n2[96];
    snprintf(n1, sizeof n1, "name=\"%s\"", field);
    snprintf(n2, sizeof n2, "name='%s'", field);
    const char *p = strstr(hay, n1);
    if (!p) p = strstr(hay, n2);
    if (!p) return 0;
    const char *q = strstr(p, "value=");
    if (!q) return 0;
    q += 6;
    char quote = *q++;
    if (quote != '"' ^ quote != '\') return 0;
    const char *e = strchr(q, quote);
    if (!e) return 0;
    size_t len = (size_t)(e - q);
    if (len ≥ max_len) return 0;
    memcpy(out, q, len);
    out[len] = '\0';
    html_unescape(out);
    return 1;
}
```

62. *auth_fetch* performs one step of the sign-in on a handle whose cookie store persists across the whole sequence. With *follow* zero it stops at a redirect and reports the **Location** in *redir*, which is how steps 2 and 3 obtain the next URL. The returned body is a heap string the caller frees; Λ means the request did not complete.

⟨Sign-in transport 62⟩ ≡

```
static char *auth_fetch(CURL *curl, const char *url, const char *post, int follow, char *redir, size_t
    rsz, long *status)
{
    Buffer buf = {Λ, 0};
    struct curl_slist *hdrs = Λ;
    ⟨Configure the sign-in request 63⟩ CURLcode rc = curl_easy_perform(curl);
    ⟨Collect the sign-in response 64⟩
}
```

See also section 65.

This code is used in section 57.

63. The form encoding is the one both ASP.NET endpoints expect. Setting `CURLOPT_HTTPGET` explicitly when there is no body matters because the handle is reused: without it the previous step's POST would persist.

⟨Configure the sign-in request 63⟩ ≡

```

if (post) hdrs = curl_slist_append(hdrs, "Content-Type: application/x-www-form-urlencoded");
curl_easy_setopt(curl, CURLOPT_URL, url);
curl_easy_setopt(curl, CURLOPT_HTTPHEADER, hdrs);
if (post) {
    curl_easy_setopt(curl, CURLOPT_POST, 1L);
    curl_easy_setopt(curl, CURLOPT_POSTFIELDS, post);
}
else curl_easy_setopt(curl, CURLOPT_HTTPGET, 1L);
curl_easy_setopt(curl, CURLOPT_FOLLOWLOCATION, (long) follow);
curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, write_cb);
curl_easy_setopt(curl, CURLOPT_WRITEDATA, &buf);
curl_easy_setopt(curl, CURLOPT_USERAGENT, HTTP_USER_AGENT);
curl_easy_setopt(curl, CURLOPT_ACCEPT_ENCODING, "");
curl_easy_setopt(curl, CURLOPT_COOKIEFILE, "");
curl_easy_setopt(curl, CURLOPT_TIMEOUT, (long) HTTP_TIMEOUT_SECS);
curl_easy_setopt(curl, CURLOPT_CONNECTTIMEOUT, (long) HTTP_CONNECT_SECS);

```

This code is used in section 62.

64. The status and, when the request stopped at one, the redirect target are read back before the header list is released.

⟨Collect the sign-in response 64⟩ ≡

```

long code = 0;
char *loc = Λ;
curl_easy_getinfo(curl, CURLINFO_RESPONSE_CODE, &code);
curl_easy_getinfo(curl, CURLINFO_REDIRECT_URL, &loc);
if (redir ∧ rsz) snprintf(redir, rsz, "%s", loc ? loc : "");
curl_slist_free_all(hdrs);
if (status) *status = code;
if (rc ≠ CURLE_OK) {
    fprintf(stderr, "Error: sign-in step failed at %s: %s\n", url, curl_easy_strerror(rc));
    free(buf.data);
    return Λ;
}
if (¬buf.data) buf.data = calloc(1, 1);
return buf.data;

```

This code is used in section 62.

65. *form_add* appends one percent-encoded key=value pair to a form body, inserting the separator when the body is not empty.

⟨Sign-in transport 62⟩ +≡

```

static void form_add(CURL *curl, char *buf, size_t n, const char *key, const char *val)
{
    char *e = curl_easy_escape(curl, val ? val : "", 0);
    size_t len = strlen(buf);
    snprintf(buf + len, n - len, "%s%s=%s", len ? "&" : "", key, e ? e : "");
    if (e) curl_free(e);
}

```

66. *oidc_login* walks the six steps. On success it publishes `USAS_SUB_ID` and `USAS_SESSION_ID` into the environment, from where *http_request* picks them up for every later call, and returns 1.

⟨ Perform the OIDC sign-in 66 ⟩ $\equiv$

```
static int oidc_login(const char *user, const char *pass)
{
    CURL *curl = curl_easy_init();
    if (!curl) return 0;
    int ok = 0;
    char *body = malloc(16384);
    char *form = malloc(16384);
    if (body & form) ⟨ Walk the sign-in steps 67 ⟩ free(body);
    free(form);
    curl_easy_cleanup(curl);
    return ok;
}
```

This code is used in section 57.

67. The steps in order. Each failure is reported by the chunk that detects it, so a sign-in that goes wrong says *where* it went wrong rather than simply refusing to start.

⟨ Walk the sign-in steps 67 ⟩ $\equiv$

```
{
    char url[4096], ret[4096], tok[1024];
    long st = 0;
    ⟨ Fetch the sign-in form 68 ⟩ ⟨ Submit the credentials 69 ⟩ ⟨ Follow the authorization callback 70 ⟩ ⟨ Post
        the authorization code 71 ⟩ ⟨ Read the session identity 72 ⟩
}
```

This code is used in section 66.

68. Step 1. The redirects are followed to the identity provider's login page, from which the `ReturnUrl` and the antiforgery token are lifted.

⟨ Fetch the sign-in form 68 ⟩ $\equiv$

```
char *page = auth_fetch(curl, BFF_LOGIN, A, 1, A, 0, &st);
if (!page) goto done;
if (!scan_field(page, "ReturnUrl", ret, sizeof ret) ∨ !scan_field(page, "__RequestVerificationToken",
    tok, sizeof tok)) {
    fputs("Error: sign-in page did not contain the expected form\n", stderr);
    free(page);
    goto done;
}
free(page);
```

This code is used in section 67.

69. Step 2. A 302 means the password was accepted; a 200 means the form came back with a validation message, which for this endpoint means the credentials were rejected. The two cases get different messages, because “HTTP 200” is a peculiarly unhelpful way to be told that a password is wrong.

⟨Submit the credentials 69⟩ ≡

```

form[0] = '\0';
form_add(curl, form, 16384, "ReturnUrl", ret);
form_add(curl, form, 16384, "Username", user);
form_add(curl, form, 16384, "Password", pass);
form_add(curl, form, 16384, "loginButton", "login");
form_add(curl, form, 16384, "__RequestVerificationToken", tok);
page = auth_fetch(curl, IDP_LOGIN, form, 0, url, sizeof url, &st);
if (¬page) goto done;
free(page);
if (st ≠ 302 ∨ ¬url[0]) {
    if (st ≡ 200)
        fprintf(stderr, "Error: USA Swimming rejected the credentials "
            "for user \"%s\". \n\n"
            "Check USAS_USER and \"USAS_PASS. \n", user);
    else fprintf(stderr, "Error: sign-in failed for user \"%s\" "
        "(HTTP %ld%s)\n", user, st,
        http_status_text(st));
    goto done;
}

```

This code is used in section 67.

70. Step 3. The authorization callback answers with the `response_mode=form_post` page. Its four hidden fields are the authorization code and the values that bind it to this flow.

⟨Follow the authorization callback 70⟩ ≡

```

page = auth_fetch(curl, url, Λ, 0, Λ, 0, &st);
if (¬page) goto done;
char code[1024], state[4096], sess[512], iss[256];
int got = scan_field(page, "code", code, sizeof code) ∧ scan_field(page, "state", state, sizeof
    state) ∧ scan_field(page, "iss", iss, sizeof iss);
if (¬scan_field(page, "session_state", sess, sizeof sess)) sess[0] = '\0';
free(page);
if (¬got) {
    fputs("Error: authorization callback returned no code\n", stderr);
    goto done;
}

```

This code is used in section 67.

71. Step 4. Posting the code to the back-end completes the exchange and sets the session cookie. The reply is a redirect to the application root, which we do not need to follow.

⟨ Post the authorization code 71 ⟩ ≡

```

form[0] = '\0';
form_add(curl, form, 16384, "code", code);
form_add(curl, form, 16384, "state", state);
form_add(curl, form, 16384, "session_state", sess);
form_add(curl, form, 16384, "iss", iss);
page = auth_fetch(curl, BFF_SIGNIN, form, 0, Λ, 0, &st);
if (¬page) goto done;
free(page);
if (st ≠ 302 ∧ st ≠ 200) {
    fprintf(stderr, "Error: code_exchange_failed_(HTTP_%ld)\n", st);
    goto done;
}

```

This code is used in section 67.

72. Step 5. The claims come back as JSON; *sub* and *sid* are all we need, and they become the two headers every later request carries.

⟨ Read the session identity 72 ⟩ ≡

```

page = auth_fetch(curl, BFF_USERINFO, Λ, 0, Λ, 0, &st);
if (¬page) goto done;
char sub[256], sid[256];
const char *q = page;
if (st ≡ 200 ∧ scan_string("sub", &q, sub, sizeof sub)) {
    q = page;
    if (scan_string("sid", &q, sid, sizeof sid)) {
        setenv("USAS_SUB_ID", sub, 1);
        setenv("USAS_SESSION_ID", sid, 1);
        ok = 1;
    }
}
free(page);
if (¬ok) fprintf(stderr, "Error: userinfo_returned_no_session" "(HTTP_%ld)\n", st);
done: (void)0;

```

This code is used in section 67.

73. *activate_session* performs the seventh step. It is an ordinary data-hub request — by now *http_request* is sending the new **Usas-Sub-Id** and **Usas-Session-Id** — and its reply carries the signed-in user's given name, which is worth showing so the operator can see *whose* account is in use. Without this call every protected times endpoint answers 401.

⟨ Activate the session 73 ⟩ ≡

```
static int activate_session(char *who, size_t who_sz)
{
    char json[600];
    long st = 0;
    snprintf(json, sizeof json, "{\"sub\":\"%s\",\"sid\":\"%s\"}", getenv("USAS_SUB_ID"),
             getenv("USAS_SESSION_ID"));
    char *r = http_request(SEcurity_INFO, json, &st);
    if (!r) return 0;
    if (st != 200) {
        report_http_error("session_activation", SEcurity_INFO, st);
        free(r);
        return 0;
    }
    const char *p = r;
    if (who & who_sz & !scan_string("firstName", &p, who, who_sz)) snprintf(who, who_sz, "(unknown)");
    free(r);
    return 1;
}
```

This code is used in section 57.

74. *ensure_session* is the entry point *main* calls before any fetch. A session supplied in the environment is used as it stands — but it is still activated, since an unactivated session is indistinguishable from an invalid one until a times query fails. Otherwise the credentials file supplies a user name and password and the full sign-in runs.

⟨ Ensure a signed-in session 74 ⟩ ≡

```
static int ensure_session(int quiet)
{
    char who[128] = "";
    load_env_file();
    const char *sub = getenv("USAS_SUB_ID");
    const char *sid = getenv("USAS_SESSION_ID");
    if (!sub & !sid) {⟨ Sign in with the stored credentials 75 ⟩}
    if (!activate_session(who, sizeof who)) {
        fputs("Error: the data hub did not accept this session\n", stderr);
        return 0;
    }
    if (!quiet) printf("Signed in as %s.\n\n", who);
    return 1;
}
```

This code is used in section 57.

75. The credentials file must supply both a user name and a password. If it does not, say so precisely: a missing file and a file missing one field are different mistakes with different remedies.

```

⟨Sign in with the stored credentials 75⟩ ≡
  const char *user = getenv("USAS_USER");
  const char *pass = getenv("USAS_PASS");
  if (¬user ∨ ¬*user ∨ ¬pass ∨ ¬*pass) {
    ⟨Report missing credentials 76⟩return 0;
  }
  if (¬oidc_login(user, pass)) return 0;

```

This code is used in section 74.

76. The message names the file actually consulted, so the reader is not left guessing which of the two possible paths was used.

```

⟨Report missing credentials 76⟩ ≡
{
  const char *f = getenv("USAS_ENV_FILE");
  char shown[512];
  if (f ∧ *f) snprintf(shown, sizeof shown, "%s", f);
  else
    snprintf(shown, sizeof shown, "%s%s", getenv("HOME") ? getenv("HOME") : "~", ENV_FILE_DEFAULT);
  fprintf(stderr, "Error: no_USA_Swimming_credentials.\n" "Expected_USA\
    S_USER_and_USAS_PASS_in_%s\n" "or_in_the_environment). Without\
    t_them_only\n" "offline_can_retrieve_times.\n", shown);
}

```

This code is used in section 75.

77. Database connection.

78. The `store` option writes rows into the shared Postgres database `swimming` on 100.77.243.69; the `offline` option reads them back. Both reach Postgres *programmatically* via the standard client library `libpq` — no `psql` child process or wrapper script is involved (this is requirement #3 of `requirements4.txt`: “Access the Postgres database programmatically, not through scripts”).

One database, named in the program. Earlier revisions defaulted to a local `swim-times` database and let `SWIM_TIMES_PGCONNINFO` override it. That is withdrawn: the connection string is now a constant and no environment variable can redirect it. The reason is that this program is no longer the only writer. The `swimming` database is a shared store, and a run that silently wrote somewhere else — to a stale local copy, say — would look like it had worked while contributing nothing.

Its schema is not ours. The database is normalised, not flat: a swim references a *swimmer_key* and a *meet_key* rather than repeating the swimmer’s name and the meet’s name on every row, and its *event_code*, *standard_name*, and *age_group_label* columns are foreign keys into reference tables. This program adapts to that shape rather than adding a second, flatter table of its own beside it. Three consequences run through the code below: a swimmer and a meet must be *resolved to keys* before a swim can be written; a motivational standard the reference table does not know must be written as Λ ; and the primary key *swim_time_id*, which the service supplies only on the richer `GetAllTimesForFilters` feed, must be synthesised for rows that come from `BestTimes`.

```
< Database pipe 78 > ≡
  < Resolve connection string 79 > < Open and close DB connection 80 > < Database statement
    helpers 83 > < Resolve a swimmer key 85 > < Resolve a meet key 88 > < Synthesise a swim id 89 > < Insert
    one row 90 >
```

This code is used in section 8.

79. `db_conn_string` returns the `libpq` connection string. It is a constant: there is deliberately no environment override.

```
< Resolve connection string 79 > ≡
  static const char *db_conn_string(void)
  {
    return "dbname=swimming_user=postgres host=100.77.243.69 port=5432";
  }
```

This code is used in section 78.

80. `db_open` opens a `libpq` connection; `db_close` releases it. Both are idempotent. Statements are issued with `PQexecParams` as they are needed rather than prepared up front, because the write path is now several statements rather than one and the volume — a few dozen rows per run — does not repay the bookkeeping.

```
< Open and close DB connection 80 > ≡
  static PGconn*db_conn =  $\Lambda$ ; < Open DB connection 81 > < Close DB connection 82 >
```

This code is used in section 78.

81. Connection establishment. A failure is non-fatal here: the caller chooses whether to abort. In **store** mode the program continues without persisting; in **offline** mode the caller bails out.

⟨ Open DB connection 81 ⟩ ≡

```
static int db_open(void)
{
    db_conn = PQconnectdb(db_conn_string());
    if (PQstatus(db_conn) ≠ CONNECTION_OK) {
        fprintf(stderr, "Warning: libpq connection failed: %s", PQerrorMessage(db_conn));
        PQfinish(db_conn);
        db_conn = Λ;
        return 0;
    }
    return 1;
}
```

This code is used in section 80.

82. Tear-down.

⟨ Close DB connection 82 ⟩ ≡

```
static void db_close(void)
{
    if (db_conn) {
        PQfinish(db_conn);
        db_conn = Λ;
    }
}
```

This code is used in section 80.

83. Two small wrappers carry every statement below. *db_scalar* runs a query expected to yield one value and returns it as a **long long**; *db_exec* runs a statement for its effect. Both report a failure as a warning and continue: one unwritable row must not end a run.

⟨ Database statement helpers 83 ⟩ ≡

```
static long long db_scalar(const char *sql, int n, const char **p)
{
    PGresult *r = PQexecParams(db_conn, sql, n, Λ, p, Λ, Λ, 0);
    long long v = 0;
    if (PQresultStatus(r) ≡ PGRES_TUPLES_OK ∧ PQntuples(r) ≡ 1) v = strtoll(PQgetvalue(r, 0, 0), Λ, 10);
    else if (PQresultStatus(r) ≠ PGRES_TUPLES_OK)
        fprintf(stderr, "Warning: %s", PQerrorMessage(db_conn));
    PQclear(r);
    return v;
}

static void db_exec(const char *sql, int n, const char **p)
{
    PGresult *r = PQexecParams(db_conn, sql, n, Λ, p, Λ, Λ, 0);
    if (PQresultStatus(r) ≠ PGRES_COMMAND_OK)
        fprintf(stderr, "Warning: %s", PQerrorMessage(db_conn));
    PQclear(r);
}
```

This code is used in section 78.

84. Finding the swimmer already in the table. This is harder than it looks, and getting it wrong duplicates people.

The obvious lookup — *member_id* or an exact *full_name* — misses, because the two writers of this database learned their names from different feeds and spell them differently. The sibling loader recorded middle *initials*; the feed this program reads returns middle *names*. So Liliya P Fedyshyn and Liliya Penny Fedyshyn are one person under two spellings, and an exact match finds neither the row nor the collision until the **UNIQUE** constraint on *full_name* refuses the insert. Case differs too: NOVA LARUE ELLIS against Nova Larue Ellis.

So the match proceeds in three tiers, most trustworthy first:

1. the *member_id*, which is the service's own identifier and settles the question outright;
2. the full name, compared case-insensitively;
3. first and last name only, compared case-insensitively, which bridges the middle-initial difference.

Tiers 2 and 3 apply *only to rows whose member_id is Λ* . That guard is what keeps the fuzziness safe: a row already carrying a different member id is a different person, however alike the names, and merging two swimmers is a far worse outcome than creating a duplicate. An unlinked row, by contrast, has never been confirmed against the service and is exactly what tiers 2 and 3 exist to claim.

85. *db_find_swimmer* implements that. The **ORDERBY** makes the tiers explicit, so that when more than one row could match the strongest match wins.

(Resolve a swimmer key 85) $\equiv$

```
static long long db_find_swimmer(const char *member_id, const char *name)
{
    const char *p[2] = {member_id, name};
    return db_scalar("SELECT swimmer_key FROM swimmer WHERE \
        ember_id = $1 " OR (member_id IS NULL AND lower(full_n\
        ame) = lower($2)) " OR (member_id IS NULL " AND lower(split_part(full_name, ' ', \
        1)) = " AND lower(split_part($2, ' ', 1)) " AND lower(regex\
        xp_replace(full_name, '^.*\s', '')) = " AND lower(regex_replace($2, '^.*\s\\ \
        s', '')) " ORDER BY (member_id = $1) DESC, " AND (lower(fu\
        ll_name) = lower($2)) DESC LIMIT 1", 2, p);
}
```

See also sections 86 and 87.

This code is used in section 78.

86. A swimmer no tier matched is new, and is inserted. The insert can still fail, and one cause is worth naming: a row already holding this name under a *different* member id. That is two distinct swimmers registered under one name, which the table's **UNIQUE** constraint refuses and which this program must not paper over by attaching one swimmer's times to the other. The row is skipped and the conflict reported, because a person has to decide it.

(Resolve a swimmer key 85) $+\equiv$

```
static long long db_insert_swimmer(const char *member_id, const char *name, const char
    *lsc, const char *club)
{
    const char *ins[4] = {member_id, name, lsc, club};
    long long k = db_scalar("INSERT INTO swimmer (member_id, full_name, " lsc_code, club_name \
        e, last_fetched) " VALUES ($1, $2, $3, $4, now()) " RETURNING swimmer_key", 4, ins);
    if (!k) fprintf(stderr,
        "Warning: could not add swimmer \"%s\" (%s); " her times were not stored.\n",
        name, member_id ? member_id : "no member id");
    return k;
}
```

87. *db_swimmer_key* returns the key for a swimmer, creating the row if it is absent. An existing row is refreshed rather than overwritten: *coalesce* fills in a member id, LSC, or club that was previously unknown without discarding one already recorded. The recorded *full_name* is deliberately among the things left alone — a loose match means the two spellings denote one person, not that ours is the better spelling — so linking Liliya P Fedyshyn to her member id leaves her name as the table already had it.

⟨ Resolve a swimmer key 85 ⟩ $\equiv$

```
static long long db_swimmer_key(const char *member_id, const char *name, const char *lsc, const
    char *club)
{
    long long k = db_find_swimmer(member_id, name);
    if ( $\neg$ k) return db_insert_swimmer(member_id, name, lsc, club);
    char kb[32];
    snprintf(kb, sizeof kb, "%lld", k);
    const char *upd[4] = {kb, member_id, lsc, club};
    db_exec("UPDATE_swimmer SET member_id = coalesce(member_id, $2), " "lsc_code = coalesce\
        e($3, lsc_code), " "club_name = coalesce($4, club_name), la\
        st_fetched = now() " "WHERE_swimmer_key = $1::bigint", 4, upd);
    return k;
}
```

88. *db_meet_key* resolves a meet name to its key, inserting a row when the name is new. The insert supplies only *meet_name*: the service's own *meetId*, start and end dates come from the richer feed this program does not use, and leaving them Λ lets that feed fill them in later. A nameless meet yields 0, which the caller stores as a Λ *meet_key*.

⟨ Resolve a meet key 88 ⟩ $\equiv$

```
static long long db_meet_key(const char *meet)
{
    if ( $\neg$ meet  $\vee$   $\neg$ *meet) return 0;
    const char *p[1] = {meet};
    long long k = db_scalar("SELECT_meet_key FROM_meet " "WHERE_meet_name = $1 LIMIT 1", 1, p);
    if (k) return k;
    return db_scalar("INSERT INTO_meet (meet_name) VALUES ($1) " "RETURNING_meet_key", 1, p);
}
```

This code is used in section 78.

89. The synthesised swim id. *swim.swim_time_id* is the primary key and has no default, because it normally holds the service's own *swimTimeId* — which **BestTimes** does not return. A surrogate is therefore derived from the row's natural key by taking the leading sixty-three bits of its SHA-256 and negating them. Negative values cannot collide with the service's own positive ids, the mapping is stable so re-running a fetch reproduces the same id, and the hash is already in the program for the **rate-key**. The rows the sibling loader wrote from **BestTimes** use negative ids for the same reason.

⟨Synthesise a swim id 89⟩ ≡

```
static long long swim_surrogate_id(long long swimmer, const char *event, const char *t, const
    char *date, long long meet)
{
    char buf[512];
    unsigned char d[32];
    Sha256 s;

    snprintf(buf, sizeof buf, "%lld|%s|%s|%s|%lld", swimmer, event, t, date ? date : "", meet);
    sha256_init(&s);
    sha256_update(&s, buf, strlen(buf));
    sha256_final(&s, d);

    unsigned long long h = 0;
    for (int i = 0; i < 8; i++) h = (h << 8) | d[i];
    h &= #7FFFFFFFFFFFFFFFULL;
    return -(long long)(h ? h : 1);
}
```

This code is used in section 78.

90. *db_standard* maps a motivational standard onto the *time_standard* reference table, which knows the seven age-group levels and nothing else. **BestTimes** also reports elite labels — **Summer Jrs**, **Nats**, **Trials** — and writing one would violate the foreign key, so an unrecognised label becomes Λ . The label is not lost to the reader: it still appears in the printed table and the CSV, which are not constrained by anyone's schema.

⟨Insert one row 90⟩ ≡

```
static const char *db_standard(const char *s)
{
    static const char *KNOWN[] = {"A", "AA", "AAA", "AAAA", "B", "BB", "Slower_Than_B"};
    if (!s || !*s) return  $\Lambda$ ;
    for (size_t i = 0; i < sizeof KNOWN / sizeof KNOWN[0]; i++)
        if (strcmp(s, KNOWN[i]) == 0) return KNOWN[i];
    return  $\Lambda$ ;
}
```

See also section 91.

This code is used in section 78.

91. `db_insert_row` writes one `TimeRow`. The swimmer and meet are resolved to keys first — each is a query, but both are answered from cache after the first row of a run — and the swim itself is then inserted. `ONCONFLICTDONOTHING` without a target absorbs *any* unique violation, which covers both the natural key (this swim is already recorded) and the primary key (an improbable surrogate collision), so a re-run is a no-op rather than a page of warnings.

```

⟨Insert one row 90⟩ +≡
static void db_insert_row(const char *swimmer, const char *event, const TimeRow *r)
{
    if (!db_conn) return;
    long long sk = db_swimmer_key(g_cur_member, swimmer, g_cur_lsc[0] ? g_cur_lsc : Λ,
                                  g_cur_club[0] ? g_cur_club : Λ);
    if (!sk) return;
    long long mk = db_meet_key(r->meet);
    ⟨Bind the swim row 92⟩ db_exec(⟨The swim insert statement 93⟩, 9, vals);
}

```

92. Every parameter is passed as text and cast in SQL, which keeps the binding uniform; an absent value is a `Λ` pointer rather than an empty string, so the `::date` and `::float8` casts never see one.

```

⟨Bind the swim row 92⟩ ≡
char idb[32], skb[32], mkb[32], secb[64];
snprintf(idb, sizeof idb, "%lld", swim_surrogate_id(sk, event, r->time, r->date, mk));
snprintf(skb, sizeof skb, "%lld", sk);
snprintf(mkb, sizeof mkb, "%lld", mk);
snprintf(secb, sizeof secb, "%g", r->sort_key);
const char *vals[9] = {idb, skb, event, mk ? mkb : Λ, r->date[0] ? r->date : Λ, r->time,
                      r->sort_key > 0 ? secb : Λ, db_standard(r->standard), g_cur_club[0] ? g_cur_club : Λ};

```

This code is used in section 91.

93. The statement records where the row came from. `source_endpoint` and `auth_mode` are columns the shared schema requires, and they are worth having: a reader can tell a best time scraped from `BestTimes` from a full result set loaded by the sibling program, and can tell which rows were gathered before the data hub required a sign-in.

```

⟨The swim insert statement 93⟩ ≡
"INSERT INTO swim (swim_time_id, swimmer_key, event_code, meet_key, " "swim_date, swim_ti\
me, seconds, standard_name, club_name, " "source_endpoint, a\
uth_mode) VALUES ($1::bigint, $2::bigint, $3, $4::bi\
gint, $5::date, $6, " "$7::float8, $8, $9, 'BestTimes', 'authentic\
ated') ON CONFLICT DO NOTHING"

```

This code is used in section 91.

94. Person lookup. Resolving a swimmer to her *memberId* now has two paths. When the roster already carries a *member_id* we confirm it with `GET /GetMember/<id>`, which the data hub still answers for anonymous callers. Otherwise we fall back to `POST /GetMembersForFilters`, which it does not: that call is the first place an unauthenticated run now fails, and *report_http_error* says so rather than leaving the user with a bare “request failed”.

95. Both lookups return a member record carrying the swimmer’s club and LSC just after her name, and the shared *swimmer* table has a column for each. *capture_club_and_lsc* lifts them from wherever the caller’s scan has reached; a record that omits either leaves the corresponding buffer empty, which *db_insert_row* passes on as a SQL Λ .

```

⟨ Person lookup 95 ⟩ ≡
static void capture_club_and_lsc(const char *p)
{
    g_cur_club[0] = '\0';
    g_cur_lsc[0] = '\0';
    if (!p) return;
    const char *q = p;
    scan_string("clubName", &q, g_cur_club, sizeof g_cur_club);
    q = p;
    scan_string("lscCode", &q, g_cur_lsc, sizeof g_cur_lsc);
}

```

See also sections 96 and 100.

This code is used in section 8.

96. *lookup_by_member_id* takes the known identifier at face value and uses the lookup only to recover the swimmer’s registered full name for display. A non-200 is reported and treated as a failure so a stale or mistyped id does not silently produce an empty report.

```

⟨ Person lookup 95 ⟩ +≡
static char *lookup_by_member_id(const Swimmer *sw, const char **out_name)
{
    char url[512];
    long status = 0;
    snprintf(url, sizeof url, "%s/GetMember/%s", TIMES_API, sw->member_id);
    g_not_found = 0;
    char *resp = http_request(url,  $\Lambda$ , &status);
    if (!resp) return  $\Lambda$ ;
    ⟨ Check the member lookup status 97 ⟩ ⟨ Read the member record 98 ⟩
}

```

97. An id the directory does not hold is the by-id counterpart of the search's 404: a fact about this roster entry, not a failure of the service. Anything else is a failure and stops the run.

⟨ Check the member lookup status 97 ⟩ ≡

```

if (status ≡ 404) {
    fprintf(stderr, "Warning: no member with id %s\n", sw-member-id);
    g_not_found = 1;
    free(resp);
    return Λ;
}
if (status ≠ 200) {
    report_http_error("member_lookup", url, status);
    free(resp);
    return Λ;
}

```

This code is used in section 96.

98. The record yields the registered full name for display and, just after it, the club and LSC the shared *swimmer* table records.

⟨ Read the member record 98 ⟩ ≡

```

char full[256];
const char *p = resp;
if (out_name) *out_name = scan_string("fullName", &p, full, sizeof full) ? strdup(full) : Λ;
capture_club_and_lsc(p);
free(resp);
return strdup(sw-member-id);

```

This code is used in section 96.

99. *lookup_member_id* posts a member search for *sw-search_query* and scans the result records for one whose full name contains *sw-match_substr* (after lower-casing). It returns the matching swimmer's *memberId* as a heap string (an alphanumeric token such as 6CD35348E5824C, *not* the old numeric PersonKey) and, optionally, the full name in **out_name*. Returns Λ on failure. The function body is split into six sub-modules.

100.

⟨ Person lookup 95 ⟩ +≡

```

static char *lookup_member_id(const Swimmer *sw, const char **out_name) {⟨ Take the
    known-memberId short cut 101 ⟩⟨ Build person-search URL 102 ⟩⟨ Build person-search
    body 103 ⟩⟨ Issue person-search request 104 ⟩⟨ Scan person-search result 105 ⟩⟨ Return
    person-search result 107 ⟩}

```

101. A roster entry that already knows its identifier never touches the closed search endpoint.

⟨ Take the known-memberId short cut 101 ⟩ ≡

```

if (sw-member_id ∧ *sw-member_id) return lookup_by_member_id(sw, out_name);

```

This code is used in section 100.

102. The URL addresses the member-search endpoint.

⟨ Build person-search URL 102 ⟩ ≡

```

char url[512];
snprintf(url, sizeof url, "%s/GetMembersForFilters", TIMES_API);

```

This code is used in section 100.

103. The request body is a *SFPersonSearchFilters* object. The original code sent only the free-text *name* field; we now send the complete four-field filter the data-hub front end sends — *orgCode*, *lscCode*, *isCurrent*, *name* — so that a body-shape check can never be mistaken for the authorisation failure we are actually diagnosing. The service matches *name* against member full names, so a distinctive surname keeps the result set small.

```
< Build person-search body 103 > ≡
char body[512];
snprintf(body, sizeof body,
    "{ \"orgCode\":null, \"lscCode\":null, \"isCurrent\":1, \"name\":\"%s\" }",
    sw->search_query);
```

This code is used in section 100.

104. The HTTP POST is issued. A Λ response means the request never completed (*http_request* has already said why).

A 404 needs care, and getting it wrong was a defect. The member search answers 404 **Not Found** when *no member matches the name* — which is a perfectly ordinary outcome for a roster entry whose query has gone stale, not a failure of the service. Treating it as one aborted the entire roster walk at the first such entry and silently skipped every swimmer after it. It now sets *g_not_found* and returns Λ , which the dispatcher reads as “skip this one”. Any other non-200 is a genuine failure and still stops the run.

This is the same mistake, in the same shape, as the one **BestTimes** produced: a status that means “there is no such thing” read as a status that means “something went wrong”. Each distinct status deserves its own decision.

```
< Issue person-search request 104 > ≡
long status = 0;
g_not_found = 0;
char *resp = http_request(url, body, &status);
if (!resp) return  $\Lambda$ ;
if (status == 404) {
    fprintf(stderr, "Warning: no USA Swimming member matches \"%s\"\n", sw->search_query);
    g_not_found = 1;
    free(resp);
    return  $\Lambda$ ;
}
if (status != 200) {
    report_http_error("member_search", url, status);
    free(resp);
    return  $\Lambda$ ;
}
```

This code is used in section 100.

105. The response is a JSON array of member objects. Records are scanned until one whose lower-cased full name contains *match_substr* is found; its *memberId* is duplicated. The loop body is a separate sub-chunk so the outer scanner stays inside the twenty-four line limit.

```
< Scan person-search result 105 > ≡
char *key =  $\Lambda$ ;
char *name =  $\Lambda$ ;
const char *p = resp;
while (p) { < Match one person row 106 > }
```

This code is used in section 100.

106. Within each record *memberId* precedes *fullName*, so we read the id first, then the name; if the name matches we keep the id already in hand.

⟨ Match one person row 106 ⟩ ≡

```

char member[64];
if ( $\neg$ scan_string("memberId",&p,member,sizeof member)) break;
char full_name[256];
if ( $\neg$ scan_string("fullName",&p,full_name,sizeof full_name)) break;
char lower[256];
size_t flen = strlen(full_name);
for (size_t i = 0; i ≤ flen; i++)
    lower[i] = (char)(full_name[i] ≥ 'A' ∧ full_name[i] ≤ 'Z' ? full_name[i] + 32 : full_name[i]);
if (strstr(lower,sw→match_substr)) {
    key = strdup(member);
    name = strdup(full_name);
    capture_club_and_lsc(p);
    break;
}

```

This code is used in section 105.

107. The response buffer is freed, a diagnostic is printed when nothing matched, and the *memberId* (or Λ) is returned. A result set that contains no row whose name holds *match_substr* is the same kind of outcome as a 404: the service answered, and this swimmer is not in the answer. It sets *g_not_found* too.

⟨ Return person-search result 107 ⟩ ≡

```

free(resp);
if ( $\neg$ key) {
    fprintf(stderr,"Warning: no search result for \"%s\" contains \"%s\"\n",sw→search_query,
            sw→match_substr);
    g_not_found = 1;
}
if (out_name) *out_name = name;
else free(name);
return key;

```

This code is used in section 100.

108. Times fetch.

109. *insertion_sort* sorts n **TimeRow** records in-place by *sort_key* ascending (smallest = fastest time). A swimmer’s career rarely exceeds a few dozen entries per event, so $O(n^2)$ is entirely adequate.

```

⟨Times fetch 109⟩ ≡
static void insertion_sort(TimeRow *rows, int n)
{
    for (int i = 1; i < n; i++) {
        TimeRow tmp = rows[i];
        int j = i - 1;
        while (j ≥ 0 ∧ rows[j].sort_key > tmp.sort_key) {
            rows[j + 1] = rows[j];
            j--;
        }
        rows[j + 1] = tmp;
    }
}

```

See also sections 110, 112, 113, 114, 115, 117, and 119.

This code is used in section 8.

110. *time_to_seconds* converts a formatted swim time ("22.64r", "1:40.36", "14:59.62") to a **double** number of seconds, used as the sort key. Colon-separated groups are accumulated sexagesimally; any trailing flag character (such as the **r** marking a relay lead-off) is ignored.

```

⟨Times fetch 109⟩ +=
static double time_to_seconds(const char *t)
{
    double total = 0;
    char buf[32];
    size_t bi = 0;
    for (const char *s = t; ; s++) {
        if (*s == ':' ∨ *s == '\0') {
            buf[bi] = '\0';
            total = total * 60 + strtod(buf, &);
            bi = 0;
            if (*s == '\0') return total;
        }
        else if (((*s ≥ '0' ∧ *s ≤ '9') ∨ *s == '.') ∧ bi < sizeof buf - 1) {
            buf[bi++] = *s;
        }
    }
}

```

111. Retrieving date, meet, and standard. The lightweight **GetBestTimesForMember** feed this program originally used returned only a stroke, distance, course, and formatted time — no swim date, meet name, or motivational standard. Those three fields are, however, reachable anonymously through the richer **POST /BestTimes** query, the same call the data-hub front-end issues to render a swimmer’s “Best Times” page. Given a *memberId*, a stroke abbreviation, and a distance, **BestTimes** returns the swimmer’s lifetime-best time in each course (SCY and LCM) for that stroke-and-distance pair, and each record now carries *meetName*, *swimDate*, *eventCode*, *swimTime*, and *timeStandard* — the B, BB, A, AA, AAA, AAAA, or elite (Nats, Trials, ...) level attained. This is requirement #1 and #2 of **requirements6.txt**.

112. Because **BestTimes** keys on a single stroke-and-distance pair, one POST is issued per distinct pair among the requested events (about twenty-two calls for the full thirty-one-event roster, since each call delivers both courses). Every parsed record is stored in a per-member cache *g_cache*; each per-event query is then answered from that cache without further network traffic. The *date_min* and *date_max* age-window parameters are retained for signature compatibility but are not applied online.

⟨Times fetch 109⟩ +≡

```
typedef struct {
    char event[32];
    TimeRow row;
} CacheEntry;
static char g_cache_member[64] = ""; /* memberId the cache holds */
static CacheEntry g_cache[NUM_EVENTS]; /* one best row per event */
static int g_cache_n = 0; /* live entries in g_cache */
```

113. *normalize_date* converts the API's "Mon DD, YYYY" date (e.g. "Jan 19, 2019") to ISO YYYY-MM-DD. The ISO form sorts lexically, contains no comma (so it needs no CSV quoting), and matches the format the offline read path reconstructs with *to_char*. An unrecognised input is copied through unchanged.

⟨Times fetch 109⟩ +≡

```
static void normalize_date(const char *in, char *out, size_t n)
{
    static const char *M = "JanFebMarAprMayJunJulAugSepOctNovDec";
    char mon[4];
    int day = 0, yr = 0;
    if (sscanf(in, "%3s_%d_%d", mon, &day, &yr) ≠ 3) {
        snprintf(out, n, "%s", in);
        return;
    }
    const char *pos = strstr(M, mon);
    int m = pos ? (int)((pos - M)/3) + 1 : 0;
    if (yr < 0 ∨ yr > 9999) yr = 0; /* bound so ISO output fits 16 bytes */
    if (m < 0 ∨ m > 99) m = 0;
    if (day < 0 ∨ day > 99) day = 0;
    snprintf(out, n, "%04d-%02d-%02d", yr, m, day);
}
```

114. *cache_add* appends one parsed **BestTimes** record to *g_cache*, computing the numeric sort key from the formatted time and normalising the date to ISO form.

⟨Times fetch 109⟩ +≡

```
static void cache_add(const char *event, const char *time, const char *date, const char
    *standard, const char *meet)
{
    if (g_cache_n ≥ NUM_EVENTS) return;
    CacheEntry *e = &g_cache[g_cache_n++];
    snprintf(e-event, sizeof e-event, "%s", event);
    e-row.sort.key = time_to_seconds(time);
    snprintf(e-row.time, sizeof e-row.time, "%s", time);
    normalize_date(date, e-row.date, sizeof e-row.date);
    snprintf(e-row.standard, sizeof e-row.standard, "%s", standard);
    snprintf(e-row.meet, sizeof e-row.meet, "%s", meet);
}
```

115. *fetch_pair* POSTs one **BestTimes** query for a stroke abbreviation and distance and parses every returned record into the cache. It returns 1 when the query succeeded and 0 when it did not, so that *build_member_cache* can abandon the remaining twenty-one pairs instead of replaying a refusal once per stroke and distance.

One status needs care. **BestTimes** answers 404 **Not Found** when the swimmer has simply never swum that stroke at that distance — and for a ten-year-old that is most of the thirty-one events, the 1000 and 1650 free and the 400 IM among them. A 404 is therefore data, not failure: the pair contributes no rows and the walk continues. Treating it as an error, as an earlier draft of this revision did, aborted the default all-events run on the first event the swimmer had never entered.

⟨Times fetch 109⟩ +≡

```
static int fetch_pair(const char *member_id, const char *stroke, long dist)
{
    char url[256], body[256];
    long status = 0;
    snprintf(url, sizeof url, "%s/BestTimes", TIMES_API);
    snprintf(body, sizeof body,
        "{\n\"memberId\": \"%s\", \"strokeAbbreviation\": \"%s\", \"distance\": %ld}",
        member_id, stroke, dist);
    char *resp = http_request(url, body, &status);
    if (!resp) return 0;
    if (status == 404) {
        free(resp);
        return 1;
    }
    if (status != 200) {
        report_http_error("best-times_query", url, status);
        free(resp);
        return 0;
    }
    ⟨Parse BestTimes records 116⟩ free(resp);
    return 1;
}
```

116. Within each record the fields appear in the fixed order *meetName*, *eventCode*, *swimDate*, *swimTime*, *timeStandard*, so the scanner reads them in that sequence, advancing its position pointer past each. *timeStandard* may be JSON *null* (no standard earned); the scan then leaves *std* empty.

⟨Parse BestTimes records 116⟩ ≡

```
const char *p = resp;
char meet[256], event[32], date[16], stime[32], std[48];
while (scan_string("meetName", &p, meet, sizeof meet)) {
    if (!scan_string("eventCode", &p, event, sizeof event)) break;
    if (!scan_string("swimDate", &p, date, sizeof date)) break;
    if (!scan_string("swimTime", &p, stime, sizeof stime)) break;
    std[0] = '\0';
    scan_string("timeStandard", &p, std, sizeof std);
    cache_add(event, stime, date, std, meet);
}
```

This code is used in section 115.

117. *build_member_cache* fills *g_cache* for *member_id* by issuing one **BestTimes** POST per distinct stroke-and-distance pair drawn from the effective event list (the **-e** selection when given, otherwise all thirty-one **EVENTS**). Pairs already fetched are skipped so each POST runs at most once.

⟨Times fetch 109⟩ +≡

```
static int build_member_cache(const char *member_id) { g_cache_n = 0;
    int use_e = (g_nevents > 0);
    int n = use_e ? g_nevents : NUM_EVENTS;
    char done[NUM_EVENTS][16];
    int nd = 0; for (int i = 0; i < n; i++) { const char *code = use_e ? g_events[i] : EVENTS[i];
    long dist;
    char stroke[8], key[16];
    if (sscanf(code, "%ld_%7s", &dist, stroke) ≠ 2) continue;
    snprintf(key, sizeof key, "%s_%ld", stroke, dist); ⟨Skip pair if already fetched 118⟩
    if (¬fetch_pair(member_id, stroke, dist)) return 0;
    } snprintf(g_cache_member, sizeof g_cache_member, "%s", member_id);
    return 1; }
```

118. A linear scan of the *done* list suppresses a repeat POST for a stroke-and-distance pair whose SCY and LCM events both appear in the event list.

⟨Skip pair if already fetched 118⟩ ≡

```
int seen = 0;
for (int j = 0; j < nd; j++)
    if (strcmp(done[j], key) ≡ 0) {
        seen = 1;
        break;
    }
if (seen) continue;
if (nd < NUM_EVENTS) snprintf(done[nd++], sizeof done[0], "%s", key);
```

This code is used in section 117.

119. *fetch_times* reports *member_id*'s best time in *event_code*. The per-member cache is rebuilt whenever the requested member changes; the matching row (if any) is then copied out and handed to the shared sort/emit chunk. It returns 0 if the cache could not be built, which propagates the data hub's refusal up to *main* instead of printing thirty-one empty event tables.

⟨Times fetch 109⟩ +≡

```
static int fetch_times(const char *member_id, const char *event_code, const char *swimmer_name, int
    opts, const char *date_min, const char *date_max)
{
    (void) date_min;
    (void) date_max;
    if (strcmp(g_cache_member, member_id) ≠ 0)
        if (¬build_member_cache(member_id)) return 0;
    ⟨Select cached time for event 120⟩⟨Sort and emit rows 121⟩return 1;
}
```

120. The cache holds at most one best row per event, so the selection is a simple linear scan for the matching *event_code*.

⟨Select cached time for event 120⟩ ≡

```

TimeRow rows[MAX_TIMES];
int nrows = 0;

for (int i = 0; i < g_cache_n ∧ nrows < MAX_TIMES; i++)
    if (strcmp(g_cache[i].event, event_code) == 0) rows[nrows++] = g_cache[i].row;

```

This code is used in section 119.

121. The sort runs once and then up to three emitters are dispatched according to the option mask: CSV to stdout, DB pipe write, or the table to stdout. CSV and DB emit can fire together; **store** without **csv** writes silently to the database while still printing the table.

⟨Sort and emit rows 121⟩ ≡

```

insertion_sort(rows, nrows);

int lim = (opts & OPT_FASTEST) ? (nrows > 0 ? 1 : 0) : nrows;

if (opts & OPT_CSV) {⟨Emit rows as CSV 122⟩}
if ((opts & OPT_STORE) ∧ db_conn) {⟨Emit rows to DB 123⟩}
if (¬(opts & OPT_CSV)) {⟨Emit rows as table 124⟩}

```

This code is used in sections 119 and 127.

122. The CSV form on standard output is the same six-column layout honoured by the existing CAST report pipeline.

⟨Emit rows as CSV 122⟩ ≡

```

for (int i = 0; i < lim; i++)
    printf("%s\\", "%s\\", "%s\\", "%s\\", "%s\\", "%s\\", swimmer_name ? swimmer_name : "",
           event_code, rows[i].time, rows[i].date, rows[i].standard, rows[i].meet);

```

This code is used in section 121.

123. The DB form delegates each row to *db_insert_row*. Per-row inserts let ON CONFLICT DO NOTHING swallow duplicates one at a time without aborting the surrounding work — the bulk COPY path would have rolled back the whole batch on the first collision.

⟨Emit rows to DB 123⟩ ≡

```

for (int i = 0; i < lim; i++) db_insert_row(swimmer_name ? swimmer_name : "", event_code, &rows[i]);

```

This code is used in section 121.

124. The table form is the human-readable per-event block printed when **csv** is not requested.

⟨Emit rows as table 124⟩ ≡

```

printf("%s_--_%s:\\n", event_code,
       (opts & OPT_FASTEST) ? "fastest_time" : "all_times_(fastest_first)");
printf("%-12s_%-10s_%-13s_\\n", "Time", "Date", "Standard", "Meet");
printf("%-12s_%-10s_%-13s_\\n", "-----", "-----", "-----", "----");
for (int i = 0; i < lim; i++)
    printf("%-12s_%-10s_%-13s_\\n", rows[i].time, rows[i].date, rows[i].standard, rows[i].meet);
if (nrows == 0) printf("(no_times_found)\\n");
putchar('\\n');

```

This code is used in section 121.

125. Offline fetch.

126. *offline_fetch* is the read counterpart to the insert path: it issues a parameterised `SELECT` via `libpq` on the shared *db_conn* and feeds each tuple into the same **TimeRow** array consumed by the shared sort/emit chunk. No network is touched and no child process is spawned. Because the database is shared, this path reports *every* swim recorded for the swimmer in that event — including the far larger set the sibling loader gathers from `GetAllTimesForFilters` — and not merely the best times this program itself has written. The function takes a **Swimmer** pointer so it can use both *match_substr* (for an `ILIKE` clause that maps the user keyword to whichever full name the database happens to hold) and the optional age window (*date_min*, *date_max*).

```
< Offline fetch 126 > ≡
  < Define offline_fetch 127 >
```

This code is used in section 8.

127. The fetch function is decomposed into three sub-chunks so each stays inside the twenty-four line limit.

```
< Define offline_fetch 127 > ≡
  static void offline_fetch(const Swimmer *sw, const char *event_code, int opts) { < Build offline query
    params 128 > < Read offline rows from libpq 129 > < Sort and emit rows 121 > }
```

This code is used in section 126.

128. The query uses two positional parameters: the `ILIKE` pattern (`%match_substr%`) and the event code. `libpq` quotes them safely so no shell-quoting hazards apply.

The shape of the query is the visible face of the schema change. A swim carries keys, not names, so the swimmer is reached by a join and the meet by an *outer* join — *meet_key* is nullable, and a swim whose meet was never recorded must still be reported. *coalesce* turns the nullable text columns into empty strings, which is what the row copier and the emitters already expect, and *to_char* renders the date in the ISO form the rest of the program uses. Rows with no recorded time in seconds sort last rather than first.

```
< Build offline query params 128 > ≡
  char like_pat[256];
  snprintf(like_pat, sizeof like_pat, "%%s%%", sw->match_substr);
  const char *params[2] = {like_pat, event_code};
  const char *q = "SELECT sw.full_name, s.swim_time, "coalesce(to_char(s.\
    swim_date, 'YYYY-MM-DD'), '')", "coalesce(s.standard_name, ''), "coalesce(\
    m.meet_name, '')", "coalesce(s.seconds, 0)" "FROM swim s" "JOIN swimmer sw ON \
    sw.swimmer_key = s.swimmer_key" "LEFT JOIN meet m ON m.meet_key = s.meet\
    _key" "WHERE sw.full_name ILIKE $1 AND s.event_code = $2" "ORDER BY s.seconds \
    NULLS LAST";
```

This code is used in section 127.

129. The query is executed, the result is iterated, and each tuple is copied into a **TimeRow**. The age window is honoured here so that an offline query that ignores the filter (e.g. a wider DB range) still produces age-appropriate output.

```

⟨ Read offline rows from libpq 129 ⟩ ≡
    char swimmer_buf[256] = "";
    const char *swimmer_name = swimmer_buf;
    TimeRow rows[MAX_TIMES];
    int nrows = 0;
    if (¬db_conn) {
        fputs("Error: no DB connection for offline read\n", stderr);
        return;
    }
    PGresult *res = PQexecParams(db_conn, q, 2, Λ, params, Λ, Λ, 0);
    if (PQresultStatus(res) ≠ PGRES_TUPLES_OK) {
        fprintf(stderr, "Error: offline query failed: %s", PQerrorMessage(db_conn));
        PQclear(res);
        return;
    }
    int total = PQntuples(res);
    for (int i = 0; i < total ∧ nrows < MAX_TIMES; i++) {⟨ Copy one libpq row 130 ⟩}
        PQclear(res);

```

This code is used in section 127.

130. Each tuple carries six columns matching the **SELECT** list: swimmer name, time, date, standard, meet, sort key. Rows that fall outside the swimmer's optional age window are skipped without incrementing *nrows*.

```

⟨ Copy one libpq row 130 ⟩ ≡
    const char *f0 = PQgetvalue(res, i, 0);
    const char *f1 = PQgetvalue(res, i, 1);
    const char *f2 = PQgetvalue(res, i, 2);
    const char *f3 = PQgetvalue(res, i, 3);
    const char *f4 = PQgetvalue(res, i, 4);
    const char *f5 = PQgetvalue(res, i, 5);
    if (sw-date_min ∧ strcmp(f2, sw-date_min) < 0) continue;
    if (sw-date_max ∧ strcmp(f2, sw-date_max) > 0) continue;
    if (¬*swimmer_buf) {
        strncpy(swimmer_buf, f0, sizeof swimmer_buf - 1);
        swimmer_buf[sizeof swimmer_buf - 1] = '\0';
    }
    strncpy(rows[nrows].time, f1, sizeof rows[nrows].time - 1);
    rows[nrows].time[sizeof rows[nrows].time - 1] = '\0';
    strncpy(rows[nrows].date, f2, sizeof rows[nrows].date - 1);
    rows[nrows].date[sizeof rows[nrows].date - 1] = '\0';
    strncpy(rows[nrows].standard, f3, sizeof rows[nrows].standard - 1);
    rows[nrows].standard[sizeof rows[nrows].standard - 1] = '\0';
    strncpy(rows[nrows].meet, f4, sizeof rows[nrows].meet - 1);
    rows[nrows].meet[sizeof rows[nrows].meet - 1] = '\0';
    rows[nrows].sort_key = strtod(f5, Λ);
    nrows++;

```

This code is used in section 129.

131. Main program.

132. We define a small **Swimmer** record to hold the person-search parameters for each swimmer. Each entry supplies a unique *id* string (the token the user passes to `-o` to select her), a *search_query* string sent to the database (ideally distinctive enough to return a small set), and a *match_substr* (lower-cased) used to identify the correct row among the results. The optional *date_min* and *date_max* fields restrict output to swims whose date falls in $[date_min, date_max]$ (inclusive, ISO YYYY-MM-DD). These are used to express age-group windows—for example Katie Ledecky’s 9- and 10-year-old swims fall between her ninth and eleventh birthdays. Kalea’s entry searches “Benavente” because the simple two-word query “Kalea Benavente” returns no results; the API requires an exact substring match against the registered name “Kalea Rose Benavente”. Kenneth’s entry uses “kenneth ray” and Keith’s uses “keith santiago” to avoid false matches on the common surname “Evans”. Both queries have since stopped matching anything at all — the search now answers 404 for “Ray Evans” and “Santiago Evans” — so both rows carry their *memberId* and no longer search. This is the general remedy for a roster entry whose name query has gone stale, and it is cheaper besides: one GET instead of a POST and a scan. Katie Ledecky was born 17 March 1997, so the window 2006-03-17 through 2008-03-16 captures every swim from her ninth birthday up to (but not including) her eleventh. The same construction yields 2008-03-17 through 2010-03-16 for the 11- and 12-year-old window (`ledecky12`) and 2010-03-17 through 2012-03-16 for the 13- and 14-year-old window (`ledecky14`). Her *memberId* is a matter of public record, so the three Ledecky rows carry it in the sixth field and skip the member search entirely; the remaining rows leave *member_id* implicitly Λ and are resolved by name.

⟨Main function 132⟩ ≡

```
#define LEDECKY_ID DIAG_MEMBER    /* Katie Ledecky’s public memberId */
static const Swimmer SWIMMERS[] = {⟨Family and Ledecky roster rows 133⟩⟨CAST roster rows 134⟩};
#define NUM_SWIMMERS ((int)(sizeof SWIMMERS/sizeof SWIMMERS[0]))
```

See also sections 135, 136, 137, 146, 147, 148, 149, 151, 152, and 156.

This code is used in section 8.

133. The four family swimmers the program began with, followed by Katie Ledecky’s three age-group windows. Only the Ledecky rows supply a *member_id*; the rest are resolved by name.

⟨Family and Ledecky roster rows 133⟩ ≡

```
{ "stella", "Julianna_Evans", "stella",  $\Lambda$ ,  $\Lambda$ , { "kalea", "Benavente", "kalea",  $\Lambda$ ,  $\Lambda$  }, { "kenny",
  "Ray_Evans", "kenneth_ray",  $\Lambda$ ,  $\Lambda$ , "09562A75A1E646" }, { "keith", "Santiago_Evans",
  "keith_santiago",  $\Lambda$ ,  $\Lambda$ , "13A0A6894A4940" }, { "ledecky10", "Ledecky", "katie", "2006-03-17",
  "2008-03-16", LEDECKY_ID }, { "ledecky12", "Ledecky", "katie", "2008-03-17", "2010-03-16",
  LEDECKY_ID }, { "ledecky14", "Ledecky", "katie", "2010-03-17", "2012-03-16", LEDECKY_ID },
```

This code is used in section 132.

134. The College Area Swim Team roster (all LSC SI). The query is “First Last”; the match token is a lower-case substring that uniquely picks the CAST member among the search results — chosen to sidestep nicknames and same-name swimmers in other LSCs. Stella J. Evans is the same swimmer as *stella* above and so is not repeated. This chunk is split from the previous one only to honour the twenty-four-line module limit.

⟨ CAST roster rows 134 ⟩ ≡

```
{ "bothwell-emma", "EmmaBothwell", "emma", Λ, Λ, { "coombs-wally", "WallyCoombs", "wally", Λ,
Λ, { "cruz-eva", "EvaCruz", "cruz_rael", Λ, Λ, { "darrow-zoe", "ZoeDarrow", "zoe", Λ, Λ,
{ "doan-eva", "EvaDoan", "eva_doan", Λ, Λ, { "ellis-nova", "NovaEllis", "nova", Λ, Λ,
{ "fedyshyn-liliya", "LiliyaFedyshyn", "liliya", Λ, Λ, { "glass-layla", "LaylaGlass",
"layla", Λ, Λ, { "glass-logan", "LoganGlass", "wiley", Λ, Λ, { "gosser-hannah",
"HannahGosser", "gosser", Λ, Λ, { "huynh-diamond", "DiamondHuynh", "diamond", Λ, Λ,
{ "knuth-hannah", "HannahKnuth", "knuth", Λ, Λ, { "lefebvre-bailey-cleo",
"CleoLefebvreBailey", "cleo", Λ, Λ, { "mendez-giavanna", "GiavannaMendez", "mendez", Λ,
Λ, { "mulvaney-sofia", "SofiaMulvaney", "sofia", Λ, Λ, { "shay-naia", "NaiaShay", "naia",
Λ, Λ, { "simmons-briar", "BriarSimmons", "briar", Λ, Λ, { "simmons-presley",
"PresleySimmons", "aspen", Λ, Λ, { "soto-lucas", "LucasSoto", "erick", Λ, Λ,
{ "soto-vivienne", "VivienneSoto", "vivi", Λ, Λ, { "wittmershaus-addison",
"AddisonWittmershaus", "addison", Λ, Λ, { "zahner-elsie", "ElsieZahner", "elsie", Λ, Λ }
```

This code is used in section 132.

135. *parse_opts_str* tokenises a comma-separated option string (the argument to -o) and sets bits in *g_opts*. Unknown tokens are silently ignored so that future options can be added without breaking existing invocations.

⟨ Main function 132 ⟩ +=

```
static void parse_opts_str(const char *s)
{ /* Work on a writable copy so strtok can insert NUL bytes. */
  char *buf = strdup(s);
  if (!buf) return;
  char *tok = strtok(buf, ",");
  while (tok) {
    if (strcmp(tok, "fastest") == 0) g_opts |= OPT_FASTEST;
    else if (strcmp(tok, "csv") == 0) g_opts |= OPT_CSV;
    else if (strcmp(tok, "store") == 0) g_opts |= OPT_STORE;
    else if (strcmp(tok, "offline") == 0) g_opts |= OPT_OFFLINE;
    else if (strcmp(tok, "diag") == 0) g_opts |= OPT_DIAG;
    else if (strcmp(tok, "selftest") == 0) g_opts |= OPT_SELFTEST;
    else if (g_nsel < MAX_SEL) g_sel[g_nsel++] = strdup(tok);
    tok = strtok(Λ, ",");
  }
  free(buf);
}
```

136. *parse_events_str* tokenises a comma-separated event-code string (the argument to *-e*) and appends each code to *g_events*. Codes that would overflow the *NUM_EVENTS*-entry array are silently dropped. The flag may be supplied multiple times; each invocation extends the list.

⟨Main function 132⟩ +≡

```
static void parse_events_str(const char *s)
{
    char *buf = strdup(s);
    if (!buf) return;
    char *tok = strtok(buf, ",");
    while (tok & g_nevents < NUM_EVENTS) {
        /* Trim leading spaces left by comma-split with spaces around commas. */
        while (*tok == ' ') tok++;
        strncpy(g_events[g_nevents], tok, sizeof g_events[0] - 1);
        g_events[g_nevents][sizeof g_events[0] - 1] = '\0';
        g_nevents++;
        tok = strtok(NULL, ",");
    }
    free(buf);
}
```

137. *print_usage* writes the full usage message to standard error. It is split into two sub-modules: one for the options section and one for the examples section.

⟨Main function 132⟩ +≡

```
static void print_usage(const char *prog) {⟨Print usage options 138⟩⟨Print usage examples 144⟩}
```

138. The options section is split into three sub-modules so each *fprintf* stays inside the twenty-four line limit.

⟨Print usage options 138⟩ ≡

```
⟨Print usage flags 139⟩⟨Print usage swimmers 142⟩⟨Print usage event codes 143⟩
```

This code is used in section 137.

139. The *-o* keyword list. A token is either one of the four behaviour keywords below or a swimmer *id*; behaviour keywords and ids may be freely mixed. With no swimmer id every swimmer on the roster is processed.

⟨Print usage flags 139⟩ ≡

```
⟨Print the option keywords 140⟩⟨Print the sign-in note 141⟩
```

This code is used in section 138.

140. The behaviour keywords and the two flags.

⟨Print the option keywords 140⟩ ≡

```
fprintf(stderr,
    "Usage: %s -o token[,token...] [-e event[,event...]]\n\n" " -o token,... c\
omma-separated; each token is a behaviour keyword\n" " o\
r a swimmer id. Behaviour keywords: \n" " fastest print only the singl\
e fastest time per event\n" " csv emit CSV output (hea\
der + one line per time)\n" " store persist each row to\
Postgres swim-times via libpq\n" " offline read times from the\
Postgres swim-times DB\n" " instead of querying\
USA Swimming over the network\n" " diag probe data-hub reach\
ability and exit\n" " selftest run the SHA-256/HMAC known-answer tests\
and exit\n" " Any other token selects a swimmer by id; with none, all a\
re shown. \n\n" " -m memberId fetch this data-hub memberId directly, by\
passing\n" " the roster and the name search\n\n", prog);
```

This code is used in section 139.

141. Where the credentials come from, which is the first thing a new operator needs to know and the only thing they must set up.

⟨Print the sign-in note 141⟩ ≡

```
fputs("Sign-in: every online run signs in to the USA Swim\
ming\n" " data hub first; there\
is no anonymous access.\n" " Credentials are read\
from $USAS_ENV_FILE, or\n" " $HOME/.usas-env, which\
should contain:\n" " export USAS_USER=\
\"...\"\" export USAS_PASS=\"...\"\" U\
SAS_SUB_ID and USAS_SESSION_ID, if both are\n" " \
et, are used instead of signing in;\n" " USAS_DEVICE_ID pins t\
he Device-Id header.\n" " Only -o offline needs no credentials at a\
ll.\n\n", stderr);
```

This code is used in section 139.

142. The swimmer ids are listed straight from the SWIMMERS roster so the help text can never drift out of sync with the table.

⟨Print usage swimmers 142⟩ ≡

```
fprintf(stderr, "swimmer ids (%d): \n", NUM_SWIMMERS);
for (int i = 0; i < NUM_SWIMMERS; i++) fprintf(stderr, "%s%-22s", (i % 3 == 0) ? "      " : "",
    SWIMMERS[i].id, (i % 3 == 2 || i == NUM_SWIMMERS - 1) ? "\n" : " ");
fputc('\n', stderr);
```

This code is used in section 138.

143. The `-e` event-code menu.

$$\langle \text{Print usage event codes } 143 \rangle \equiv$$

```
fprintf(stderr, "%%-event, ... restrict output to one or more event code\
s;\n" " %%% comma-separated, or repeat -e for each ev\
ent.\n" " %%% SCY codes: %50_FR_SCY, %100_FR_SCY, %200_FR_SCY, %500_F\
R_SCY, \n" " %%% %1000_FR_SCY, %1650_FR_SCY, \n" " %%% %50_FL_SCY, %100_FL_SCY, %50_BK_SCY, %100_BK_SCY, \n" " %%% %50_BR_SCY, %100_BR_SCY, %100_IM_SCY, %200_IM_SCY\n" " %%% LCM codes: %50\
50_FR_LCM, %100_FR_LCM, %200_FR_LCM, %400_FR_LCM, \n" " %%% %800_FR_LCM, %1500_FR_LCM, \n" " %%% %50_FL_LCM, %100_FL_LC\
M, %200_FL_LCM, \n" " %%% %50_BK_LCM, %100_BK_LC\
M, %200_BK_LCM, \n" " %%% %50_BR_LCM, %100_BR_LC\
M, %200_BR_LCM, \n" " %%% %200_IM_LCM, %400_IM_LCM\n\n");
```

This code is used in section 138.

144. The examples section illustrates common invocations.

$$\langle \text{Print usage examples } 144 \rangle \equiv$$

```
fprintf(stderr,
    "Examples:\n"
    "\"%%s-%%ustella,fastest\n"
    "\"%%s-%%okalea,csv\n"
    "\"%%s-%%okenny-e-\\\"100_FR_SCY\"\\n"
    "\"%%s-%%keith-e-\\\"100_FR_SCY,50_FL_SCY\"\\n"
    "\"%%s-%%stella-e-\\\"100_FR_SCY\"-e-\\\"50_FL_SCY\"\\n"
    "\"%%s-%%stella,csv,store#####_fetch\\_+\\_print\\_+\\_persist\\_to\\_DB\n"
    "\"%%s-%%kalea,offline#####_#_read\\_from\\_DB,\\_no\\_network\n"
    "\"%%s-%%ledecky10,fastest#####_#_KatieLedecky\\_as\\_a\\_9-10\\_y\\_r\\_old\n"
    "\"%%s-%%diag#####_#_which_endpoints_can_this\\_host_reach?\n"
    "\"%%s-m6CD35348E5824C#####_#_fetch_one_memberId_directly\n",
    prog, prog, prog, prog, prog, prog, prog, prog, prog, prog);
```

This code is used in section [137](#).

145. Known-answer self-test. The rate-key a signed-in caller presents is only as good as the hash underneath it, and a wrong hash would surface as an opaque 403 — indistinguishable from the lockout this program already has to explain. `-o selftest` therefore checks SHA-256 and HMAC-SHA256 against the published vectors of FIPS 180-4 and RFC 4231 before anyone has to trust them. It performs no I/O beyond printing its verdict.

146. The expected digests, kept together so the vectors can be checked against the standards at a glance.

$$\langle \text{Main function } 132 \rangle + \equiv$$

```
#define FOX "The_quick_brown_fox_jumps_over_the_lazy_dog"
#define SHA_EMPTY "e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855"
#define SHA_ABC "ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad"
#define SHA_A199 "60048478ae47edd7ef18f1235afd254a72ffaf32c4bc5726e8d250c3be51e3cb"
#define HMAC_EMPTY "b613679a0814d9ec772f95d778c35fc5ff1697c493715653c6c712144292c5ad"
#define HMAC_FOX "f7bc83f430538424b13298e6aa6fb143ef4d59a14946175997479dbc2d1a3cd8"
```

147. *check_vector* compares one computed digest with its expected value, prints a one-line verdict, and returns the boolean so the caller can accumulate an overall result.

```

⟨Main function 132⟩ +=
static int check_vector(const char *label, const char *got, const char *want)
{
    int ok = (strcmp(got, want) == 0);
    printf("%%-28s%%s\\n", label, ok ? "ok" : "FAILED");
    if (!ok) printf("%%s\\n%%s\\n%%s\\n", want, got);
    return ok;
}

```

148. *sha256_hex* is a convenience wrapper that digests one null-terminated string into a hexadecimal buffer, mirroring the output format *hmac_sha256* already produces.

```

⟨Main function 132⟩ +=
static void sha256_hex(const char *msg, char *hex)
{
    Sha256 s;
    unsigned char d[32];
    sha256_init(&s);
    sha256_update(&s, msg, strlen(msg));
    sha256_final(&s, d);
    for (int i = 0; i < 32; i++) snprintf(hex + i * 2, 3, "%02x", d[i]);
}

```

149. The vectors: the empty string and "abc" from FIPS 180-4, a 200-byte message that spans four compression blocks and exercises the padding path, and two HMAC cases — the empty key over the empty message, and RFC 4231's "quick brown fox".

```

⟨Main function 132⟩ +=
static int run_selftest(void)
{
    char h[65];
    int ok = 1;

    printf("swim-times_cryptographic_self-test\\n\\n");
    sha256_hex("", h);
    ok &= check_vector("SHA-256(\\\"\\\")", h, SHA_EMPTY);
    sha256_hex("abc", h);
    ok &= check_vector("SHA-256(\\\"abc\\\")", h, SHA_ABC);
    ⟨Check the multi-block vector 150⟩hmac_sha256("", "", h);
    ok &= check_vector("HMAC-SHA256(\\\"\\\",\\\"\\\")", h, HMAC_EMPTY);
    hmac_sha256("key", FOX, h);
    ok &= check_vector("HMAC-SHA256(\\\"key\\\",fox)", h, HMAC_FOX);
    printf("\\n%%s\\n", ok ? "all_vectors_passed" : "SELF-TEST_FAILED");
    return ok ? RC_OK : RC_ERROR;
}

```

150. A 199-byte message crosses the 64-byte block boundary three times and lands in the awkward region where the length field forces an extra padding block.

⟨ Check the multi-block vector 150 ⟩ ≡

```
char big[200];
memset(big, 'a', sizeof big - 1);
big[sizeof big - 1] = '\0';
sha256_hex(big, h);
ok &= check_vector("SHA-256(\"a\"_x_199)", h, SHA_A199);
```

This code is used in section 149.

151. Diagnostics. `-o diag` answers the question the old error message could not: is the data hub down, is the network broken, or has this caller simply been refused? It prints the identity being presented and then probes one endpoint from each access class, so the three cases are told apart at a glance — all 200 means a working, authorised client; 200 on the reference feeds with 403 on the queries is the anonymous lockout; anything else is a genuine outage.

⟨ Main function 132 ⟩ +≡

```
static void probe_endpoint(const char *label, const char *url, const char *body)
{
    long status = 0;
    char *r = http_request(url, body, &status);
    printf("_%-38s_%-4s_%3ld_%%s\n", label, body ? "POST" : "GET", status, http_status_text(status));
    free(r);
}
```

152. `run_diagnostics` reports the credentials in force and then walks the probe list.

⟨ Main function 132 ⟩ +≡

```
static int run_diagnostics(void)
{
    printf("swim-times_data-hub_diagnostics\n\n");
    ⟨ Report the sign-in attempt 153 ⟩ printf("__Usas-Sub-Id_____:%%s\n", usas_sub_id());
    printf("__Usas-Session-Id_:%%s\n", usas_session_id() ? "(set)" : "(none)");
    printf("__Device-Id_____:%%s\n\n", device_id());
    ⟨ Probe data-hub endpoints 155 ⟩ ⟨ Explain the probe results 154 ⟩ return RC_OK;
}
```

153. Diagnosis is the one place the program proceeds after a failed sign-in. Reporting which endpoints an *unauthenticated* caller can still reach is exactly what the operator needs in order to tell a bad password from an expired session from an outage, so *run_diagnostics* records the outcome and probes either way.

```

⟨Report the sign-in attempt 153⟩ ≡
    char who[128] = "";
    int signed_in = 0;
    load_env_file();
    if (getenv("USAS_USER") ∨ getenv("USAS_SUB_ID")) {
        signed_in = ensure_session(1);
        if (signed_in ∧ activate_session(who, sizeof who)) printf("Sign-in:ok\n", who);
        else if (signed_in) printf("Sign-in:ok\n");
        else printf("Sign-in:FAILED---probing_unauthenticated\n");
    }
    else {
        printf("Sign-in:not_attempted(no_credentials_found)\n");
    }

```

This code is used in section 152.

154. The closing note reads differently depending on whether the sign-in worked, because the same status means different things in the two cases.

```

⟨Explain the probe results 154⟩ ≡
    if (signed_in)
        printf("\nAll five should read 200 for a signed-in caller.\n" "A401 here means \
            the session was not activated;\n" "a403 means this account lacks the pe\
            rmission.\n");
    else
        printf("\nThe last two are expected to fail without a sign-in:\n" "403=refused as \
            anonymous, 401=session not recognised.\n" "Fix the credentia\
            ls in the file named above and re-run.\n");

```

This code is used in section 152.

155. Two reference feeds and the single-member lookup are open to anonymous callers; the member search and the best-times query are the two calls this program depends on and the two that are now closed.

```

⟨Probe data-hub endpoints 155⟩ ≡
    char url[512];
    snprintf(url, sizeof url, "%s/SearchFilter/GetAllEvents", SWIMS_API);
    probe_endpoint("SearchFilter/GetAllEvents", url, Λ);
    snprintf(url, sizeof url, "%s/GetTimeStandardAgeGroups", TIMES_API);
    probe_endpoint("TimesSearch/GetTimeStandardAgeGroups", url, Λ);
    snprintf(url, sizeof url, "%s/GetMember/%s", TIMES_API, DIAG_MEMBER);
    probe_endpoint("TimesSearch/GetMember/<id>", url, Λ);
    snprintf(url, sizeof url, "%s/GetMembersForFilters", TIMES_API);
    probe_endpoint("TimesSearch/GetMembersForFilters", url,
        "{ \"orgCode\":null, \"lscCode\":null, \"isCurrent\":1, \"name\":\"Leducky\" }");
    snprintf(url, sizeof url, "%s/BestTimes", TIMES_API);
    probe_endpoint("TimesSearch/BestTimes", url,
        "{ \"memberId\":\"DIAG_MEMBER\", \"strokeAbbreviation\":\"FR\", \"distance\":50 }");

```

This code is used in section 152.

156. We initialise the global `libcurl` state, parse the optional `-o`, `-e`, and `-m` flags, then for each swimmer (subject to the selected swimmer ids in `g_sel`) resolve her `memberId` and iterate over events. If one or more `-e` codes were given only those events are fetched; otherwise all thirty-one are processed. In CSV mode a single header line is printed before the first data row. If no options at all are supplied, the usage message is printed and the program exits with status 1. The function body is split into three sub-modules.

⟨Main function 132⟩ +≡

```
int main(int argc, char *argv[]) {⟨Parse command-line options 157⟩⟨Check for empty
    invocation 158⟩⟨Fetch and print swimmer times 161⟩}
```

157. The option-parsing loop processes `-o`, `-e`, and `-m` flags via `getopt`.

⟨Parse command-line options 157⟩ ≡

```
int ch;
while ((ch = getopt(argc, argv, "o:e:m:")) ≠ -1) {
    switch (ch) {
        case 'o': parse_opts_str(optarg);
            break;
        case 'e': parse_events_str(optarg);
            break;
        case 'm': g_member_id = optarg;
            break;
        default: print_usage(argv[0]);
            return RC_USAGE;
    }
}
```

This code is used in section 156.

158. If no options were supplied the usage message is printed and the program exits. Otherwise global curl state is initialised (skipped under `offline`, where no HTTP is performed), the CSV header is emitted if needed, and the DB pipe is opened if `store` was requested.

⟨Check for empty invocation 158⟩ ≡

```
if (g_opts ≡ 0 ∧ g_nevents ≡ 0 ∧ g_nsel ≡ 0 ∧ ¬g_member_id) {
    print_usage(argv[0]);
    return RC_USAGE;
}
if (¬(g_opts & OPT_OFFLINE)) curl_global_init(CURL_GLOBAL_DEFAULT);
⟨Run diagnostics and exit 159⟩⟨Sign in before fetching 160⟩
if (g_opts & OPT_CSV)
    printf("\nSwimmer\\", "Event\\", "Time\\", "Date\\", "Standard\\", "Meet\\n");
if (g_opts & (OPT_STORE | OPT_OFFLINE)) {
    int ok = db_open();
    if (¬ok ∧ (g_opts & OPT_OFFLINE)) {
        fputs("Error: _offline_mode_requires_a_DB_connection\n", stderr);
        return RC_UNAVAIL;
    }
}
```

This code is used in section 156.

159. `-o diag` is a self-contained mode: it touches no database and fetches no swimmer, so it reports and returns before any of that is set up. It is meaningless under `offline`, where the network is never used, and says so.

```

⟨Run diagnostics and exit 159⟩ ≡
  if (g_opts & OPT_SELFTEST) {
    int t = run_selftest();
    if (¬(g_opts & OPT_OFFLINE)) curl_global_cleanup();
    return t;
  }
  if (g_opts & OPT_DIAG) {
    if (g_opts & OPT_OFFLINE) {
      fputs("Error: _diag_and_offline_are_mutually_exclusive\n", stderr);
      return RC_USAGE;
    }
    int d = run_diagnostics();
    curl_global_cleanup();
    return d;
  }

```

This code is used in section 158.

160. Every online run signs in first. There is no anonymous path: the data hub answers a search or a times query from an unauthenticated caller with `403` and an empty body, so attempting one would only waste a round trip on the way to a failure the program can already predict. Under `offline` no network is used and no sign-in is attempted. The greeting is suppressed in CSV mode, where a line of prose on standard output would corrupt the file.

```

⟨Sign in before fetching 160⟩ ≡
  if (¬(g_opts & OPT_OFFLINE) ∧ ¬ensure_session(g_opts & OPT_CSV)) {
    curl_global_cleanup();
    return RC_NOPERM;
  }

```

This code is used in section 158.

161. Each swimmer is visited in turn. The swimmer-filter mask is applied before the expensive *memberId* lookup. `-m` replaces the roster with a single synthetic entry carrying just the requested *memberId*, which is how a swimmer who is not on the roster — or one whose name search is now refused — can still be fetched. The DB connection (if any) is closed before `libcurl` is torn down, and the run's exit status distinguishes a refusal (`RC_NOPERM`) from any other failure.

```

⟨Fetch and print swimmer times 161⟩ ≡
  const Swimmer adhoc = {"member", "", "", Λ, Λ, g_member_id};
  const Swimmer *roster = g_member_id ? &adhoc : SWIMMERS;
  int nroster = g_member_id ? 1 : NUM_SWIMMERS;
  int rc = RC_OK;
  int found_any = 0;
  for (int s = 0; s < nroster; s++) {⟨Process one swimmer 163⟩}
  ⟨Report skipped swimmers 162⟩db_close();
  if (¬(g_opts & OPT_OFFLINE)) curl_global_cleanup();
  return rc;

```

This code is used in section 156.

162. Skipped swimmers are counted rather than merely warned about, because a warning in the middle of thirty screens of times is a warning nobody reads. The tally is repeated at the end, where it is the last thing on the terminal. A run in which *nothing* resolved is a failure — there is no output and no reason to pretend otherwise — while a run that fetched some swimmers and skipped others succeeded, with a qualification the operator can act on.

```

⟨ Report skipped swimmers 162 ⟩ ≡
  if (g_missing > 0) fprintf(stderr, "\n%d swimmer%s skipped: not found in the\
    e_USA_Swimming" "directory.\n" "Check the search query and match substr\
    in the SWIMMERS" "roster, or\ngive the swimmer a known member_id.\n", g_missing,
    g_missing ≡ 1 ? "was" : "s_were");
  if (¬found_any ∧ g_missing > 0 ∧ rc ≡ RC_OK) rc = RC_UNAVAIL;

```

This code is used in section 161.

163. One swimmer is resolved and all requested events are fetched. Under *offline* the network *memberId* lookup is bypassed and the swimmer's display name is taken from the database in *offline_fetch*. The online branch is split into its own sub-chunk.

```

⟨ Process one swimmer 163 ⟩ ≡
  if (¬g_member_id ∧ g_nsel > 0) {
    int selected = 0;
    for (int k = 0; k < g_nsel; k++)
      if (strcmp(g_sel[k], roster[s].id) ≡ 0) {
        selected = 1;
        break;
      }
    if (¬selected) continue;
  }
  if (g_opts & OPT_OFFLINE) { ⟨ Fetch events offline 166 ⟩ continue; } ⟨ Online swimmer dispatch 164 ⟩

```

This code is used in section 161.

164. The online path resolves a *memberId* and dispatches the per-event fetcher. A lookup can fail in two ways and they are answered differently. If the swimmer simply is not in the service's directory, she is skipped and the walk goes on — one stale roster entry must not cost the other twenty-eight. If the lookup failed for any other reason, the run stops: when the data hub has refused one swimmer for want of credentials it will refuse the next identically, and a page of the same error helps nobody. The exit status carries which it was.

⟨ Online swimmer dispatch 164 ⟩ ≡

```

const char *name = Λ;
char *key = lookup_member_id(&roster[s], &name);
if (¬key) {
    if (g_not_found) {
        g_missing++;
        continue;
    }
    rc = g_auth_blocked ? RC_NOPERM : RC_UNAVAIL;
    break;
}
g_cur_member = key;
found_any = 1;
if (¬(g_opts & OPT_CSV))
    printf("Swimmer: %s (MemberId: %s)\n\n", name ? name : "(unknown)", key);
⟨ Fetch events for swimmer 165 ⟩ free(key);
free((void *) name);
if (¬(g_opts & OPT_CSV)) putchar('\n');
if (rc ≠ RC_OK) break;

```

This code is used in section 163.

165. Either the user-requested events or all thirty-one default events are fetched online. The optional age window from the **Swimmer** record is forwarded to *fetch_times*.

⟨ Fetch events for swimmer 165 ⟩ ≡

```

int nev = (g_nevents > 0) ? g_nevents : NUM_EVENTS;
for (int i = 0; i < nev; i++) {
    const char *code = (g_nevents > 0) ? g_events[i] : EVENTS[i];
    if (¬fetch_times(key, code, name, g_opts, roster[s].date_min, roster[s].date_max)) {
        rc = g_auth_blocked ? RC_NOPERM : RC_UNAVAIL;
        break;
    }
}

```

This code is used in section 164.

166. The offline counterpart calls *offline_fetch* once per event. No HTTP request is performed, no *memberId* is resolved, and the DB pipe is irrelevant (**store** composes only with online runs).

⟨ Fetch events offline 166 ⟩ ≡

```

if (g_nevents > 0) {
    for (int i = 0; i < g_nevents; i++) offline_fetch(&roster[s], g_events[i], g_opts);
}
else {
    for (int i = 0; i < NUM_EVENTS; i++) offline_fetch(&roster[s], EVENTS[i], g_opts);
}
if (¬(g_opts & OPT_CSV)) putchar('\n');

```

This code is used in section 163.

167. Glossary.

The following terms and interfaces appear throughout this program.

168.

USA Swimming times API. The first-party REST service at `times-api.usaswimming.org` that backs the public data hub at `data.usaswimming.org`. It replaced the Sisense JAQL analytics API this program originally used (now decommissioned for public callers). Requests are ordinary HTTP GET/POST calls returning JSON; the OpenAPI description is published at `.../swagger/v1/swagger.json`.

Data-hub authentication. The times API uses no bearer token. Every request instead carries `AppName: DataHub`, a `Device-Id`, and `Usas-Sub-Id`—the caller’s subject, or the literal `Anonymous` when not signed in. The server validates the *format* of the `Device-Id`—base64 of “<platform> - <vendor> - <fingerprint> - <millis>” with the first five base64 characters repeated after the fifteenth—and answers 400 `Invalid Device-Id format in request` when that shape is violated; `device_id` mints a conforming value at run time. A signed-in caller adds `Usas-Session-Id` and `rate-key`. This program takes `USAS_SUB_ID`, `USAS_SESSION_ID`, and an optional `USAS_DEVICE_ID` from the environment.

Anonymous lockout. The condition, in force since 2026, in which the data hub answers reference queries for an anonymous caller but refuses every search and every times query with 403 `Forbidden` and an empty body. It is the reason this program signs in, and `-o diag` is what identifies it when a sign-in has not happened.

Session activation. The step, easily missed, that makes a newly-issued session usable. After the OpenID Connect flow yields a `sub` and a `sid`, the times API still answers 401 `Unauthorized` until `POST security/auth/GetDataHubSecurity` has been called with that pair; afterwards it answers 200. The data hub’s own front end makes the call on start-up as a permissions query, so the coupling is invisible from a browser. `activate_session` performs it on every run.

Credentials file. `$USAS_ENV_FILE`, or `$HOME/.usas-env`: shell-style `NAME="value"` lines from which the program takes `USAS_USER` and `USAS_PASS`. It is parsed, never sourced, and only `USAS_`-prefixed names are honoured.

rate-key. A per-request throttling token a signed-in caller presents alongside its session id. Its value is `[t/s]`, a dot, and the hexadecimal HMAC-SHA256 of the decimal spelling of `t` keyed by the session id, where `t` is the Unix time in milliseconds divided by 10^4 and `s` is the number of seconds elapsed since midnight UTC. `build_rate_key` computes it; `-o selftest` checks the hash beneath it against the FIPS 180-4 and RFC 4231 vectors.

memberId. The service’s identifier for a swimmer, an alphanumeric token such as `6CD35348E5824C` (it replaces the old numeric `PersonKey`). A `memberId` known in advance is worth keeping: `GetMember/<id>` is still open to anonymous callers, while the search that would otherwise discover the id is not.

Exit status. 0 success; 2 a usage error; 69 (`RC_UNAVAIL`) the data hub could not be reached or failed for a reason other than authorisation; 77 (`RC_NOPERM`) the data hub refused the request for want of credentials. The last of these is the one a wrapper script should read as “sign in”, not “retry later”.

169. USA Swimming REST API Calls. Both endpoints live under <https://times-api.usaswimming.org/swims/Time> and carry `Content-Type: application/json` plus the anonymous auth headers above.

- **Member search.** POST `/GetMembersForFilters` with body `{"orgCode":null,"lscCode":null,"isCurrent":1,"name"`. Returns a JSON array of member objects, each with (among others) `memberId`, `fullName`, `clubName`, and `lscCode`. This program scans the array for the first member whose lower-cased `fullName` contains the match substring and keeps that record's `memberId`. *Closed to anonymous callers*: it now answers 403 unless `Usas-Sub-Id` names a signed-in subject.
- **Sign-in sequence.** GET `dhy-prod.usaswimming.org/bff/login` begins an OpenID Connect authorization-code flow with PKCE; POST `login.usaswimming.org/Login` submits `ReturnUrl`, `Username`, `Password`, `loginButton`, and `__RequestVerificationToken` as a URL-encoded form and answers 302 on success or 200 (the form, re-rendered) on rejection; the authorization callback it names returns an HTML `response_mode=form_post` page whose `code`, `state`, `session_state`, and `iss` are posted to `/signin-oidc`; and GET `/bff/userinfo` then returns the claims, of which `sub` and `sid` are used. All five steps share one cookie store.
- **Session activation.** POST `security-api.usaswimming.org/security/auth/GetDataHubSecurityInfoForIdp` with body `{"sub":"<sub>","sid":"<sid>"}`. Returns the signed-in user's identity (`firstName`, `memberId`, ...) and the list of application routes they may read. Its *side effect* is the reason it is called: until it has run, every protected times endpoint answers 401 for the new session.
- **Single-member lookup.** GET `/GetMember/<memberId>`. Returns one member object with `memberId`, `fullName`, `clubName`, `lscCode`, and `swimmerAge`. *Still open to anonymous callers*, which is why a roster entry that already knows its *member_id* can skip the search. Note that adding an `Accept: application/json` header to this call makes the service return the whole object re-encoded as a single JSON string; the program therefore sends no `Accept` header at all.
- **Best-times fetch.** POST `/BestTimes` with body `{"memberId":"<id>","strokeAbbreviation":"<abbr>","distance"`. Returns a JSON array with one object per course (SCY and LCM) for that stroke-and-distance pair, each carrying `eventCode`, `swimTime` (formatted), `swimDate` ("Mon DD, YYYY"), `meetName`, and `timeStandard` (B, BB, A, AA, AAA, AAAA, or an elite label; JSON null when none was earned). This is the query the data-hub "Best Times" page issues, and unlike the older `GetBestTimesForMember` GET it exposes date, meet, and standard to anonymous callers. One POST is issued per distinct stroke-and-distance pair. The still-richer per-swim endpoints (`GetAllTimesForFilters`, `GetSwimmerMeets`) require a signed-in subject and return 403 otherwise.

170. libcurl API Calls. This program uses the libcurl “easy” interface for synchronous HTTP. All functions return a *CURLcode* (zero = *CURLE_OK*) except where noted.

- `curl_global_init(flags)`.

Parameter: `flags` (`long`) — a bitmask of subsystems to initialise. This program passes *CURL_GLOBAL_DEFAULT*, which enables SSL and the Windows socket layer on that platform. Must be called once before any other libcurl function. Returns a *CURLcode*; this program ignores the return value because failure is treated as fatal by the subsequent easy calls.

- `curl_global_cleanup(void)`. Releases all resources allocated by *curl_global_init*. Must be called once after all easy handles have been cleaned up. Returns nothing.
- `curl_easy_init(void)`. Allocates and returns a new easy handle (a *CURL **). Returns *NULL* on failure. Each call to *http_request* creates its own handle and destroys it before returning, so handles are never shared between requests.
- `curl_easy_setopt(handle, option, value)`.

Parameters:

- `handle` (*CURL **): the easy handle.
- `option` (*CURLOption*): a constant selecting the behaviour to configure. Options used here:
CURLOPT_URL (`char *`) — the request URL.
CURLOPT_HTTPHEADER (`struct curl_slist *`) — linked list of extra HTTP headers (*Content-Type* and the anonymous *Usas-Sub-Id*, *AppName*, *Device-Id* headers).
CURLOPT_POSTFIELDS (`char *`) — the POST body; set only when a body is supplied, which also switches the method to POST (best-times fetches are plain GETs).
CURLOPT_WRITEFUNCTION (function pointer) — callback invoked for each response chunk; signature `size_t cb(void*, size_t, size_t, void*)`.
CURLOPT_WRITEDATA (`void *`) — the user-data pointer passed as the fourth argument to the write callback; here a pointer to the **Buffer** accumulator.
CURLOPT_USERAGENT (`char *`) — the *User-Agent* header; this program identifies itself truthfully rather than impersonating a browser.
CURLOPT_FOLLOWLOCATION (`long`) — non-zero to follow 3xx redirects automatically.
CURLOPT_ACCEPT_ENCODING (`char *`) — the empty string requests every content encoding libcurl was built with, and libcurl decompresses transparently.
CURLOPT_TIMEOUT (`long`) — seconds allowed for the whole transfer. Without it a hung connection could stall a run indefinitely, which is how the original version behaved.
CURLOPT_CONNECTTIMEOUT (`long`) — seconds allowed for the connect phase alone.
CURLOPT_COOKIEFILE (`char *`) — enables the cookie engine. The empty string enables it without reading a file, which is what the sign-in sequence wants: its five steps must share a cookie store, and nothing should be written to disk.
CURLOPT_POST / *CURLOPT_HTTPGET* (`long`) — select the method explicitly. This matters only because the sign-in handle is reused across steps: without an explicit *CURLOPT_HTTPGET* a following step would inherit the previous step’s POST.
- `value`: type depends on `option` (see above).

- `curl_easy_perform(handle)`.

Parameter: `handle` (*CURL **). Executes the configured request synchronously, invoking the write callback for each received chunk. Returns *CURLE_OK* on success or a non-zero error code; on failure *http_request* frees the partial buffer and returns *NULL*. Note that a 403 response is *not* a failure by this measure: the transfer succeeded, and the status must be read separately with *curl_easy_getinfo*. Conflating the two is the defect this revision repairs.

- `curl_easy_getinfo(handle, info, ...)`.

Parameters:

- `handle` (*CURL **): the easy handle, after *curl_easy_perform* and before *curl_easy_cleanup*.
- `info` (*CURLINFO*): the datum wanted. This program asks for *CURLINFO_RESPONSE_CODE* and, during sign-in, *CURLINFO_REDIRECT_URL* — the *Location* a request would have followed had redirect-following been enabled. The string it yields belongs to the handle and must not be freed.

- ...: a pointer to storage of the type that **info** implies — here a **long ***. Returns a **CURLcode**. The **long** is left untouched when no response was received, so it is initialised to zero first.

- **curl_easy_escape(handle, string, length)**.

Parameters:

- **handle** (**CURL ***): any easy handle.
- **string** (**const char ***): the text to encode.
- **length** (**int**): its length, or 0 to use **strlen**. Returns a newly-allocated percent-encoded copy, or **NULL**. The caller releases it with *curl_free*, *not free*. Used by *form_add* to build the two sign-in form bodies, whose values contain **&**, **/**, **+**, and **=** in abundance.

- **curl_free(ptr)**.

Parameter: **ptr** (**void ***) — memory returned by a libcurl function. Releases it using the allocator libcurl was built with, which need not be the one this program links against.

- **curl_easy_strerror(code)**.

Parameter: **code** (**CURLcode**). Returns a static, human-readable description of a libcurl error code. Used so a transport failure names itself (“Could not resolve host”) instead of being reported generically.

- **curl_easy_cleanup(handle)**.

Parameter: **handle** (**CURL ***). Releases all resources associated with the handle. The handle must not be used after this call.

- **curl_slist_append(list, string)**.

Parameters:

- **list** (**struct curl_slist ***): existing list head, or **NULL** to start a new list.
- **string** (**const char ***): the string to append. Returns the new list head, or **NULL** on allocation failure. Used to build the data-hub header list (**Content-Type**, **AppName**, **Origin**, **Referer**, **Usas-Sub-Id**, **Device-Id**, and, for a signed-in caller, **Usas-Session-Id** and **rate-key**). Note that libcurl does not copy the strings until the transfer runs, so the caller’s buffers must outlive the list.

- **curl_slist_free_all(list)**.

Parameter: **list** (**struct curl_slist ***). Frees every node in the linked list. Called immediately after *curl_easy_perform* so the headers are released before the handle.

171. POSIX System Calls and Library Functions. The following identifiers from the POSIX.1-2008 standard are used directly in this program. Each entry gives the C signature, a description of each parameter, and a note on how the program uses it.

- `int fprintf(FILE *stream, const char *fmt, ...)`.

Parameters:

- `stream`: destination file (`stderr` here).
- `fmt`: printf-style format string.
- `...`: values substituted into `fmt`. Writes formatted output to `stream`; returns the character count or a negative value on error. Used to report lookup and HTTP failures to standard error.

- `int fputs(const char *s, FILE *stream)`.

Parameters:

- `s`: null-terminated string to write.
- `stream`: destination file (`stderr` here). Writes `s` without a trailing newline; returns non-negative on success or `EOF` on error. Used for fixed error messages where no formatting is needed.

- `void free(void *ptr)`.

Parameter:

- `ptr`: pointer to a heap block, or `NULL` (in which case nothing happens). Releases the block back to the heap. Called on every heap string (response buffers, duplicated names and keys) when they are no longer needed.

- `void *calloc(size_t n, size_t size)`.

Parameters:

- `n`: number of elements.
- `size`: bytes per element. Allocates `n×size` bytes, zero-filled, and returns a pointer to them, or `NULL` on failure. Used in *http_request* to manufacture a one-byte, zero-filled buffer — an empty C string — when a successful response carried no body, so that `NULL` can mean “no response” and nothing else.

- `int fclose(FILE *stream)`.

Parameter: `stream` — an open stream. Flushes and closes it; returns 0 or `EOF`. Closes the credentials file.

- `char *fgets(char *s, int n, FILE *stream)`.

Parameters:

- `s`: destination buffer.
- `n`: its size; at most `n-1` bytes are read.
- `stream`: source stream. Reads up to and including the next newline, null-terminating the result; returns `s`, or `NULL` at end of file or on error. Reads the credentials file one assignment at a time.

- `FILE *fopen(const char *path, const char *mode)`.

Parameters:

- `path`: the file to open.
- `mode`: “r” here. Returns a stream, or `NULL` if the file cannot be opened — which for the credentials file is not an error in itself, since the values may instead be present in the environment.

- `char *getenv(const char *name)`.

Parameter:

- `name`: the environment-variable name. Returns a pointer to the variable’s value, or `NULL` when it is not set. The returned string belongs to the environment and must not be freed. Used to read `USAS_SUB_ID`, `USAS_SESSION_ID`, `USAS_DEVICE_ID`, and `SWIM_TIMES_PGCONNINFO` — every piece of deployment-specific configuration the program accepts.

- `struct tm *gmtime_r(const time_t *t, struct tm *result)`.

Parameters:

- `t`: pointer to a calendar time.
- `result`: caller-supplied `struct tm` to fill. Breaks `t` down into UTC calendar components and returns `result`, or `NULL` on error. Unlike `gmtime` it keeps no static state and is therefore thread-safe. Used by *build_rate_key* to obtain the seconds elapsed since midnight UTC, the divisor in the *rate-key* formula.

- `void *memcpy(void *dst, const void *src, size_t n)`.

Parameters:

- **dst**: destination address.
 - **src**: source address.
 - **n**: number of bytes to copy. Copies exactly **n** bytes from **src** to **dst**; regions must not overlap. Returns **dst**. Used in *write_cb* to append each network chunk to the buffer, and in *scan_string* to copy a JSON string value.
 - **void *memset(void *dst, int c, size_t n).**
- Parameters:
- **dst**: destination address.
 - **c**: fill byte (converted to **unsigned char**).
 - **n**: number of bytes to fill. Returns **dst**. Used to zero-pad the HMAC key block to sixty-four bytes, which RFC 2104 requires, and to build the self-test's 199-byte multi-block message.
- **int printf(const char *fmt, ...).**
- Parameters:
- **fmt**: printf-style format string.
 - **...**: values substituted into **fmt**. Writes formatted output to standard output; returns the character count or a negative value on error. Used for all swimmer and time-table output.
- **int putchar(int c).**
- Parameter:
- **c**: character value (as **unsigned char** cast to **int**). Writes one character to standard output; returns the character written, or EOF on error. Used to emit a blank line ('**\n**') after each event section.
- **int setenv(const char *name, const char *value, int overwrite).**
- Parameters:
- **name**: the variable to set; must contain no **=**.
 - **value**: its new value.
 - **overwrite**: when zero, an existing variable is left untouched. Returns 0, or -1 on error. Used twice: with **overwrite** zero to publish the credentials file without displacing anything already in the environment, and with **overwrite** one to publish the subject and session id the sign-in obtained, from where *http_request* reads them.
- **int strncmp(const char *a, const char *b, size_t n).**
- Parameters:
- **a, b**: the strings to compare.
 - **n**: the maximum number of bytes to compare. Returns a negative, zero, or positive value as **a** sorts before, equal to, or after **b** within the first **n** bytes. Used to recognise the **export** keyword and the **USAS_** prefix when parsing the credentials file, and the HTML entities in *html_unescape*.
- **time_t time(time_t *tloc).**
- Parameter:
- **tloc**: optional address to store the result in, or NULL. Returns the current time as seconds since the Epoch, or (**time_t**)-1 on failure. Used for the Device-Id fingerprint and for the **rate-key** epoch bucket.
- **void *realloc(void *ptr, size_t size).**
- Parameters:
- **ptr**: existing heap block, or NULL.
 - **size**: new size in bytes. Returns a pointer to the resized block (possibly moved), or NULL if allocation fails (the original block is unchanged on failure). When **ptr** is NULL the call is equivalent to **malloc**. Used in *write_cb* to grow the response buffer incrementally as libcurl delivers each chunk.
- **int snprintf(char *buf, size_t n, const char *fmt, ...).**
- Parameters:
- **buf**: destination character array.
 - **n**: maximum bytes to write, including the null terminator.
 - **fmt**: printf-style format string.
 - **...**: values substituted into **fmt**. Writes at most **n-1** formatted characters to **buf** and always null-terminates. Returns the number of characters that would have been written had the buffer been

unlimited (so a return value $\geq n$ signals truncation). Used to assemble URL strings, JSON bodies, and the bearer-token header.

- `char *strchr(const char *s, int c)`.

Parameters:

- `s`: string to search.
- `c`: character to find (compared as `unsigned char`). Returns a pointer to the first occurrence of `c` in `s`, including the terminator if `c` is `'\0'`, or `NULL`. Used in *scan_string* to find the closing double-quote of a JSON string value.

- `char *strdup(const char *s)`.

Parameter:

- `s`: null-terminated string to duplicate. Allocates a new heap block of `strlen(s)+1` bytes, copies `s` into it, and returns the pointer; returns `NULL` on failure. Used in *lookup_member_id* to persist the swimmer's full name and memberId string across the lifetime of a query.

- `size_t strlen(const char *s)`.

Parameter:

- `s`: null-terminated string. Returns the number of bytes before the null terminator. Used to compute loop bounds when lower-casing names and to advance past a search needle in *scan_string*.

- `char *strncpy(char *dst, const char *src, size_t n)`.

Parameters:

- `dst`: destination array (at least `n` bytes).
- `src`: source string.
- `n`: maximum bytes to copy. Copies up to `n` bytes; if `src` is shorter than `n`, the remainder of `dst` is zero-filled. If `src` is at least `n` bytes long, `dst` will *not* be null-terminated. Returns `dst`. This program always writes `dst[sizeof dst - 1] = '\0'` after the call to guarantee termination. Used to copy the swimmer name and requested event codes; fixed `TimeRow` fields are filled with *snprintf* instead.

- `char *strstr(const char *hay, const char *needle)`.

Parameters:

- `hay`: string to search within.
- `needle`: substring to search for. Returns a pointer to the first occurrence of `needle` in `hay`, or `NULL` if not found. The workhorse of the JSON scanner: used to locate key names (`"memberId"`, `"fullName"`, `"swimTime"`, ...) and to advance through the raw REST response text.

- `double strtod(const char *s, char **endptr)`.

Parameters:

- `s`: string containing a floating-point number.
- `endptr`: if non-`NULL`, receives a pointer to the first character not consumed by the conversion. Returns the parsed `double`; sets `*endptr` past the converted text. Used by *time_to_seconds* to convert each colon-separated group of a formatted time (e.g. `"1:40.36"`) to a `double` sort key.

- `int sscanf(const char *s, const char *fmt, ...)`.

Parameters:

- `s`: input string to parse.
- `fmt`: scanf-style conversion string.
- `...`: pointers to receive the converted values. Reads formatted input from `s`; returns the number of input items successfully matched and assigned (which may be fewer than requested, or EOF on failure). Used by *normalize_date* to split `"Mon DD, YYYY"` into a month abbreviation, day, and year, and by *build_member_cache* to split an event code such as `"100 FR SCY"` into its numeric distance and stroke abbreviation.

172. Index.

activate_session: [73](#), [74](#), [153](#), [168](#).
adhoc: [161](#).
advised: [49](#).
age_group_label: [78](#).
Agent: [43](#).
AppName: [13](#).
argc: [156](#), [157](#).
argv: [156](#), [157](#), [158](#).
auth_fetch: [62](#), [68](#), [69](#), [70](#), [71](#), [72](#).
auth_mode: [93](#).
b: [27](#).
base64: [28](#), [41](#).
BFF_LOGIN: [14](#), [68](#).
BFF_SIGNIN: [14](#), [71](#).
BFF_USERINFO: [14](#), [72](#).
bi: [110](#).
big: [150](#).
bits: [36](#).
body: [44](#), [45](#), [66](#), [103](#), [104](#), [115](#), [151](#).
buf: [30](#), [35](#), [36](#), [43](#), [45](#), [47](#), [62](#), [63](#), [64](#), [65](#), [89](#),
[110](#), [135](#), [136](#).
Buffer: [23](#), [27](#), [45](#), [62](#), [170](#).
build_member_cache: [115](#), [117](#), [119](#), [171](#).
build_rate_key: [40](#), [46](#), [168](#), [171](#).
BY: [85](#).
B64: [28](#).
cache_add: [114](#), [116](#).
CacheEntry: [112](#), [114](#).
calloc: [47](#), [64](#).
capture_club_and_lsc: [95](#), [98](#), [106](#).
ch: [157](#).
check_vector: [147](#), [149](#), [150](#).
club: [86](#), [87](#).
coalesce: [87](#), [128](#).
code: [47](#), [64](#), [70](#), [71](#), [117](#), [165](#).
CONFLICT: [91](#).
CONNECTION_OK: [81](#).
curl: [45](#), [47](#), [62](#), [63](#), [64](#), [65](#), [66](#), [68](#), [69](#), [70](#), [71](#), [72](#).
CURL: [45](#), [62](#), [65](#), [66](#).
curl_easy_cleanup: [47](#), [66](#), [170](#).
curl_easy_escape: [65](#).
curl_easy_getinfo: [43](#), [47](#), [64](#), [170](#).
curl_easy_init: [45](#), [66](#).
curl_easy_perform: [47](#), [62](#), [170](#).
curl_easy_setopt: [45](#), [63](#).
curl_easy_strerror: [47](#), [64](#).
curl_free: [65](#), [170](#).
curl_global_cleanup: [159](#), [160](#), [161](#).
CURL_GLOBAL_DEFAULT: [158](#).
curl_global_init: [158](#), [170](#).
curl_slist: [45](#), [62](#).
curl_slist_append: [46](#), [63](#).
curl_slist_free_all: [47](#), [64](#).
CURLcode: [47](#), [62](#), [170](#).
CURLE_OK: [47](#), [64](#).
CURLINFO_REDIRECT_URL: [64](#).
CURLINFO_RESPONSE_CODE: [47](#), [64](#).
CURLOPT_ACCEPT_ENCODING: [45](#), [63](#).
CURLOPT_CONNECTTIMEOUT: [45](#), [63](#).
CURLOPT_COOKIEFILE: [63](#).
CURLOPT_FOLLOWLOCATION: [45](#), [63](#).
CURLOPT_HTTPGET: [63](#).
CURLOPT_HTTPHEADER: [45](#), [63](#).
CURLOPT_POST: [63](#).
CURLOPT_POSTFIELDS: [45](#), [63](#).
CURLOPT_TIMEOUT: [45](#), [63](#).
CURLOPT_URL: [45](#), [63](#).
CURLOPT_USERAGENT: [45](#), [63](#).
CURLOPT_WRITEDATA: [45](#), [63](#).
CURLOPT_WRITEFUNCTION: [45](#), [63](#).
d: [89](#), [148](#), [159](#).
data: [23](#), [27](#), [35](#), [43](#), [47](#), [64](#).
DataHub: [13](#).
date: [24](#), [89](#), [92](#), [114](#), [116](#), [122](#), [124](#), [130](#).
date_max: [25](#), [112](#), [119](#), [126](#), [130](#), [132](#), [165](#).
date_min: [25](#), [112](#), [119](#), [126](#), [130](#), [132](#), [165](#).
day: [113](#).
db_close: [80](#), [82](#), [161](#).
db_conn: [80](#), [81](#), [82](#), [83](#), [91](#), [121](#), [126](#), [129](#).
db_conn_string: [79](#), [81](#).
db_exec: [83](#), [87](#), [91](#).
db_find_swimmer: [85](#), [87](#).
db_insert_row: [20](#), [91](#), [95](#), [123](#).
db_insert_swimmer: [86](#), [87](#).
db_meet_key: [88](#), [91](#).
db_open: [80](#), [81](#), [158](#).
db_scalar: [83](#), [85](#), [86](#), [88](#).
db_standard: [90](#), [92](#).
db_swimmer_key: [87](#), [91](#).
dev_hdr: [46](#).
Device: [13](#), [28](#), [41](#).
device_id: [13](#), [41](#), [46](#), [152](#), [168](#).
DIAG_MEMBER: [13](#), [132](#), [155](#).
dist: [115](#), [117](#).
DO: [91](#).
done: [68](#), [69](#), [70](#), [71](#), [72](#), [117](#), [118](#).
e: [39](#), [53](#), [61](#), [65](#), [114](#).
el: [60](#).
ensure_session: [74](#), [153](#), [160](#).
ENT: [60](#).
env: [41](#).
ENV_FILE_DEFAULT: [14](#), [58](#), [76](#).

- ep0*: [32](#), [33](#).
- ep1*: [32](#), [33](#).
- eq*: [59](#).
- event*: [89](#), [91](#), [92](#), [112](#), [114](#), [116](#), [120](#).
- event_code*: [78](#), [119](#), [120](#), [122](#), [123](#), [124](#), [127](#), [128](#).
- eventCode*: [111](#), [116](#).
- EVENTS**: [11](#), [15](#), [117](#), [165](#), [166](#).
- f*: [58](#), [76](#).
- fclose*: [58](#).
- fetch_pair*: [115](#), [117](#).
- fetch_times*: [20](#), [119](#), [165](#).
- fgets*: [58](#).
- field*: [61](#).
- flen*: [106](#).
- follow*: [62](#), [63](#).
- fopen*: [58](#).
- form*: [66](#), [69](#), [71](#).
- form_add*: [65](#), [69](#), [71](#), [170](#).
- found_any*: [161](#), [162](#), [164](#).
- FOX**: [146](#), [149](#).
- fprintf*: [47](#), [49](#), [64](#), [69](#), [71](#), [72](#), [76](#), [81](#), [83](#), [86](#), [97](#), [104](#), [107](#), [129](#), [140](#), [142](#), [143](#), [144](#), [162](#).
- fputc*: [142](#).
- fputs*: [50](#), [68](#), [70](#), [74](#), [129](#), [141](#), [158](#), [159](#).
- free*: [47](#), [64](#), [66](#), [68](#), [69](#), [70](#), [71](#), [72](#), [73](#), [97](#), [98](#), [104](#), [107](#), [115](#), [135](#), [136](#), [151](#), [164](#), [170](#).
- full*: [98](#).
- full_name*: [84](#), [87](#), [106](#).
- fullName*: [106](#).
- f0*: [130](#).
- f1*: [130](#).
- f2*: [130](#).
- f3*: [130](#).
- f4*: [130](#).
- f5*: [130](#).
- g*: [40](#).
- g_auth_blocked*: [21](#), [49](#), [164](#), [165](#).
- g_cache*: [112](#), [114](#), [117](#), [120](#).
- g_cache_member*: [112](#), [117](#), [119](#).
- g_cache_n*: [112](#), [114](#), [117](#), [120](#).
- g_cur_*: [20](#).
- g_cur_club*: [21](#), [91](#), [92](#), [95](#).
- g_cur_lsc*: [21](#), [91](#), [95](#).
- g_cur_member*: [21](#), [91](#), [164](#).
- g_events*: [21](#), [117](#), [136](#), [165](#), [166](#).
- g_member_id*: [21](#), [157](#), [158](#), [161](#), [163](#).
- g_missing*: [19](#), [21](#), [162](#), [164](#).
- g_nevents*: [21](#), [117](#), [136](#), [158](#), [165](#), [166](#).
- g_not_found*: [19](#), [21](#), [96](#), [97](#), [104](#), [107](#), [164](#).
- g_nsel*: [21](#), [135](#), [158](#), [163](#).
- g_opts*: [21](#), [135](#), [158](#), [159](#), [160](#), [161](#), [163](#), [164](#), [165](#), [166](#).
- g_sel*: [21](#), [135](#), [156](#), [163](#).
- GET**: [44](#).
- getenv*: [39](#), [41](#), [58](#), [73](#), [74](#), [75](#), [76](#), [153](#).
- getopt*: [157](#).
- getpid*: [41](#).
- gmtime_r*: [40](#).
- got*: [70](#), [147](#).
- h*: [58](#), [89](#), [149](#).
- hay*: [61](#).
- hdrs*: [45](#), [46](#), [47](#), [62](#), [63](#), [64](#).
- hex*: [37](#), [40](#), [148](#).
- HMAC_EMPTY**: [146](#), [149](#).
- HMAC_FOX**: [146](#), [149](#).
- hmac_sha256*: [37](#), [40](#), [148](#), [149](#).
- html_unescape*: [60](#), [61](#), [171](#).
- HTTP_CONNECT_SECS**: [12](#), [45](#), [63](#).
- http_request*: [43](#), [44](#), [66](#), [73](#), [96](#), [104](#), [115](#), [151](#), [170](#), [171](#).
- http_status_text*: [48](#), [49](#), [69](#), [151](#).
- HTTP_TIMEOUT_SECS**: [12](#), [45](#), [63](#).
- HTTP_USER_AGENT**: [13](#), [45](#), [63](#).
- i*: [28](#), [33](#), [36](#), [37](#), [60](#), [89](#), [90](#), [106](#), [109](#), [117](#), [120](#), [122](#), [123](#), [124](#), [129](#), [142](#), [148](#), [165](#), [166](#).
- id*: [21](#), [25](#), [41](#), [132](#), [142](#), [163](#).
- Id*: [13](#), [28](#), [41](#).
- idb*: [92](#).
- IDP_LOGIN**: [14](#), [69](#).
- ih*: [37](#).
- in*: [28](#), [113](#).
- ins*: [86](#).
- insertion_sort*: [109](#), [121](#).
- ip*: [37](#).
- isCurrent*: [103](#).
- iss*: [70](#), [71](#).
- IV**: [34](#).
- j*: [109](#), [118](#).
- json*: [73](#).
- k*: [37](#), [86](#), [87](#), [88](#), [163](#).
- kb*: [87](#).
- key*: [13](#), [37](#), [53](#), [65](#), [105](#), [106](#), [107](#), [117](#), [118](#), [164](#), [165](#).
- kl*: [37](#).
- KNOWN**: [90](#).
- K256**: [31](#), [33](#).
- label*: [147](#), [151](#).
- LEDECKY_ID**: [132](#), [133](#).
- len*: [28](#), [30](#), [34](#), [35](#), [36](#), [53](#), [61](#), [65](#).
- like_pat*: [128](#).
- lim*: [121](#), [122](#), [123](#), [124](#).
- load_env_file*: [58](#), [74](#), [153](#).
- loc*: [64](#).
- lookup_by_member_id*: [96](#), [101](#).

lookup_member_id: 25, 99, [100](#), 164, 171.
lower: [106](#).
lsc: [86](#), [87](#).
lscCode: [103](#).
M: [113](#).
m: [113](#).
main: 49, 74, 119, [156](#).
malloc: 66.
match_substr: 25, 99, 105, 106, 107, 126, 128, 132.
max_len: [53](#), [61](#).
MAX_SEL: [21](#), 135.
MAX_TIMES: [12](#), 120, 129.
meet: [24](#), [88](#), [89](#), 91, [114](#), [116](#), 122, 124, 130.
meet_key: 78, 88, 128.
meet_name: 88.
meetId: 88.
meetName: 111, 116.
member: [106](#).
member_id: 25, 84, [85](#), [86](#), [87](#), 94, 96, 97, 98, 101, [115](#), [117](#), [119](#), 132, 133, 169.
memberId: 2, 13, 21, 94, 99, 105, 106, 107, 111, 132, 156, 161, 163, 164, 166, 168.
memcpy: 27, 34, 37, 53, 61.
memset: 37, 150.
mk: [91](#), 92.
mkb: [92](#).
mon: [113](#).
msg: [37](#), [148](#).
n: [27](#), [30](#), [32](#), [35](#), 40, 41, 60, 65, 83, [109](#), [113](#), [117](#).
name: [85](#), [86](#), [87](#), 103, [105](#), 106, 107, [164](#), 165.
nd: [117](#), 118.
needle: [53](#).
nev: [165](#).
nmemb: 27.
normalize_date: [113](#), 114, 171.
NOTHING: 91.
now: [40](#), [41](#).
nroster: [161](#).
nrows: [120](#), 121, 124, [129](#), 130.
null: 116.
NUM_EVENTS: [12](#), 15, 21, 112, 114, 117, 118, [136](#), 165, 166.
NUM_SWIMMERS: [132](#), 142, 161.
n1: [61](#).
n2: [61](#).
o: [28](#).
offline_fetch: 25, 126, [127](#), 163, 166.
oh: [37](#).
oidc_login: [66](#), 75.
ok: [66](#), 72, [147](#), [149](#), 150, [158](#).
ON: 91.
op: [37](#).

OPT_CSV: [18](#), 121, 135, 158, 160, 164, 166.
OPT_DIAG: [18](#), 135, 159.
OPT_FASTEST: [18](#), 121, 124, 135.
OPT_OFFLINE: [18](#), 135, 158, 159, 160, 161, 163.
OPT_SELFTEST: [18](#), 135, 159.
OPT_STORE: [18](#), 121, 135, 158.
optarg: 157.
opts: [119](#), 121, 124, [127](#).
ORDER: 85.
orgCode: 103.
out: [28](#), [36](#), 40, [53](#), [61](#), [113](#).
out_name: [96](#), 98, 99, [100](#), 101, 107.
out_sz: [28](#).
p: [27](#), [33](#), [35](#), [53](#), [58](#), [61](#), [73](#), [83](#), [85](#), [88](#), [95](#), [98](#), [105](#), [116](#).
page: [68](#), 69, 70, 71, 72.
params: [128](#), 129.
parse_events_str: [136](#), 157.
parse_opts_str: [135](#), 157.
pass: [66](#), 69, [75](#).
path: [58](#).
PGconn: [80](#).
PGRES_COMMAND_OK: 83.
PGRES_TUPLES_OK: 83, 129.
PGresult: 83, 129.
pos: [53](#), [113](#).
post: [62](#), 63.
POST: 44.
PQclear: 83, 129.
PQconnectdb: 81.
PQerrorMessage: 81, 83, 129.
PQexecParams: 80, 83, 129.
PQfinish: 81, 82.
PQgetvalue: 83, 130.
PQntuples: 83, 129.
PQresultStatus: 83, 129.
PQstatus: 81.
print_usage: [137](#), 157, 158.
printf: 74, 122, 124, 147, 149, 151, 152, 153, 154, 158, 164.
probe_endpoint: [151](#), 155.
prog: [137](#), 140, 144.
ptr: [27](#).
putchar: 124, 164, 166.
q: [61](#), [72](#), [95](#), [128](#).
quiet: [74](#).
quote: [61](#).
r: [60](#), [73](#), [91](#), [151](#).
rate: 13.
raw: [41](#).
rc: 47, 62, 64, [161](#), 162, 164, 165.
RC_ERROR: [12](#), 149.

- RC_NOPERM: [12](#), [49](#), [160](#), [161](#), [164](#), [165](#), [168](#).
 RC_OK: [12](#), [149](#), [152](#), [161](#), [162](#), [164](#).
 RC_UNAVAIL: [12](#), [158](#), [162](#), [164](#), [165](#), [168](#).
 RC_USAGE: [12](#), [157](#), [158](#), [159](#).
 realloc: [27](#).
 redir: [62](#), [64](#).
 rem: [28](#).
 REP: [60](#).
 report_http_error: [49](#), [73](#), [94](#), [97](#), [104](#), [115](#).
 res: [129](#), [130](#).
 resp: [96](#), [97](#), [98](#), [104](#), [105](#), [107](#), [115](#), [116](#).
 ret: [67](#), [68](#), [69](#).
 rk: [46](#).
 rk_hdr: [46](#).
 ror: [32](#).
 roster: [161](#), [163](#), [164](#), [165](#), [166](#).
 row: [112](#), [114](#), [120](#).
 rows: [109](#), [120](#), [121](#), [122](#), [123](#), [124](#), [129](#), [130](#).
 rsz: [62](#), [64](#).
 run_diagnostics: [152](#), [153](#), [159](#).
 run_selftest: [149](#), [159](#).
 s: [33](#), [34](#), [35](#), [36](#), [37](#), [48](#), [59](#), [60](#), [89](#), [90](#), [110](#),
[135](#), [136](#), [148](#), [161](#).
 scan_field: [61](#), [68](#), [70](#).
 scan_string: [53](#), [72](#), [73](#), [95](#), [98](#), [106](#), [116](#), [171](#).
 search_query: [25](#), [99](#), [103](#), [104](#), [107](#), [132](#).
 secb: [92](#).
 secs: [40](#).
 SECURITY_INFO: [14](#), [73](#).
 seen: [118](#).
 selected: [163](#).
 sess: [70](#), [71](#).
 Session: [13](#).
 setenv: [59](#), [72](#).
 SFPersonSearchFilters: [103](#).
 sg0: [32](#), [33](#).
 sg1: [32](#), [33](#).
 SHA_ABC: [146](#), [149](#).
 SHA_A199: [146](#), [150](#).
 SHA_EMPTY: [146](#), [149](#).
 Sha256: [30](#), [33](#), [34](#), [35](#), [36](#), [37](#), [89](#), [148](#).
 sha256_block: [33](#), [35](#), [36](#).
 sha256_final: [36](#), [37](#), [89](#), [148](#).
 sha256_hex: [148](#), [149](#), [150](#).
 sha256_init: [34](#), [37](#), [89](#), [148](#).
 sha256_update: [35](#), [36](#), [37](#), [89](#), [148](#).
 shown: [76](#).
 sid: [40](#), [46](#), [72](#), [74](#).
 sid_hdr: [46](#).
 signed_in: [153](#), [154](#).
 size: [23](#), [27](#).
 sk: [91](#), [92](#).
 skb: [92](#).
 snprintf: [37](#), [40](#), [41](#), [46](#), [53](#), [58](#), [61](#), [64](#), [65](#), [73](#),
[76](#), [87](#), [89](#), [92](#), [96](#), [102](#), [103](#), [113](#), [114](#), [115](#),
[117](#), [118](#), [128](#), [148](#), [155](#), [171](#).
 sort_key: [24](#), [92](#), [109](#), [114](#), [130](#).
 source_endpoint: [93](#).
 sql: [83](#).
 sscanf: [113](#), [117](#).
 st: [67](#), [68](#), [69](#), [70](#), [71](#), [72](#), [73](#).
 standard: [24](#), [92](#), [114](#), [122](#), [124](#), [130](#).
 standard_name: [78](#).
 state: [70](#), [71](#).
 status: [43](#), [44](#), [45](#), [47](#), [49](#), [62](#), [64](#), [96](#), [97](#), [104](#),
[115](#), [151](#).
 std: [116](#).
 stderr: [47](#), [49](#), [50](#), [64](#), [68](#), [69](#), [70](#), [71](#), [72](#), [74](#), [76](#),
[81](#), [83](#), [86](#), [97](#), [104](#), [107](#), [129](#), [140](#), [141](#), [142](#),
[143](#), [144](#), [158](#), [159](#), [162](#).
 stime: [116](#).
 strchr: [53](#), [59](#), [61](#).
 strcmp: [90](#), [118](#), [119](#), [120](#), [130](#), [135](#), [147](#), [163](#).
 strdup: [98](#), [106](#), [135](#), [136](#).
 strlen: [28](#), [37](#), [53](#), [59](#), [60](#), [65](#), [89](#), [106](#), [148](#).
 strncmp: [59](#), [60](#).
 strncpy: [130](#), [136](#).
 stroke: [115](#), [117](#).
 strstr: [53](#), [61](#), [106](#), [113](#).
 strtod: [110](#), [130](#).
 strtok: [135](#), [136](#).
 strtoll: [83](#).
 Sub: [13](#).
 sub: [72](#), [74](#).
 sub_hdr: [46](#).
 sw: [96](#), [97](#), [98](#), [99](#), [100](#), [101](#), [103](#), [104](#), [106](#),
[107](#), [127](#), [128](#), [130](#).
 swim: [89](#).
 swim_surrogate_id: [89](#), [92](#).
 swim_time_id: [78](#), [89](#).
 SWIM_TIMES_PGCONNINFO: [78](#), [171](#).
 swimDate: [111](#), [116](#).
 Swimmer: [21](#), [25](#), [96](#), [100](#), [126](#), [127](#), [132](#),
[161](#), [165](#).
 swimmer: [20](#), [89](#), [91](#), [95](#), [98](#).
 swimmer_buf: [129](#), [130](#).
 swimmer_key: [78](#).
 swimmer_name: [119](#), [122](#), [123](#), [129](#).
 SWIMMERS: [1](#), [18](#), [25](#), [132](#), [142](#), [161](#).
 SWIMS_API: [13](#), [155](#).
 swimTime: [111](#), [116](#).
 swimTimeId: [89](#).
 t: [28](#), [40](#), [89](#), [110](#), [159](#).
 tbuf: [40](#).

time: [24](#), [40](#), [41](#), [92](#), [114](#), [122](#), [124](#), [130](#).
time_standard: [90](#).
time_to_seconds: [24](#), [110](#), [114](#), [171](#).
TimeRow: [24](#), [91](#), [109](#), [112](#), [120](#), [126](#), [129](#).
TIMES_API: [11](#), [13](#), [96](#), [102](#), [115](#), [155](#).
timeStandard: [111](#), [116](#).
tm: [40](#).
tm_hour: [40](#).
tm_min: [40](#).
tm_sec: [40](#).
tmp: [109](#).
to_char: [128](#).
tok: [67](#), [68](#), [69](#), [135](#), [136](#).
total: [110](#), [129](#).
t1: [33](#).
t2: [33](#).
ud: [27](#).
uint32_t: [30](#), [31](#), [32](#), [33](#), [34](#).
uint64_t: [30](#), [36](#).
UNIQUE: [84](#), [86](#).
upd: [87](#).
url: [44](#), [45](#), [47](#), [49](#), [62](#), [63](#), [64](#), [67](#), [69](#), [70](#), [96](#),
[97](#), [102](#), [104](#), [115](#), [151](#), [155](#).
Usas: [13](#).
USAS_DEVICE_ID: [41](#), [168](#), [171](#).
USAS_ENV_FILE: [14](#).
USAS_SESSION_ID: [46](#), [66](#), [168](#), [171](#).
usas_session_id: [39](#), [46](#), [152](#).
usas_sub_id: [39](#), [46](#), [152](#).
USAS_SUB_ID: [39](#), [66](#), [168](#), [171](#).
use_e: [117](#).
user: [66](#), [69](#), [75](#).
User: [43](#).
v: [83](#).
val: [59](#), [65](#).
vals: [91](#), [92](#).
vl: [59](#).
w: [60](#).
want: [147](#).
what: [49](#).
who: [73](#), [74](#), [153](#).
who_sz: [73](#).
write_cb: [27](#), [45](#), [63](#), [171](#).
yr: [113](#).

< Activate the session [73](#) > Used in section [57](#).
 < Apply one assignment [59](#) > Used in section [58](#).
 < Bind the swim row [92](#) > Used in section [91](#).
 < Build offline query params [128](#) > Used in section [127](#).
 < Build person-search URL [102](#) > Used in section [100](#).
 < Build person-search body [103](#) > Used in section [100](#).
 < Build request headers [46](#) > Used in section [45](#).
 < CAST roster rows [134](#) > Used in section [132](#).
 < Check for empty invocation [158](#) > Used in section [156](#).
 < Check the member lookup status [97](#) > Used in section [96](#).
 < Check the multi-block vector [150](#) > Used in section [149](#).
 < Close DB connection [82](#) > Used in section [80](#).
 < Collect the sign-in response [64](#) > Used in section [62](#).
 < Configure the sign-in request [63](#) > Used in section [62](#).
 < Copy one libpq row [130](#) > Used in section [129](#).
 < Credentials file loader [58](#) > Used in section [57](#).
 < Data hub credentials [13](#) > Used in section [11](#).
 < Database pipe [78](#) > Used in section [8](#).
 < Database statement helpers [83](#) > Used in section [78](#).
 < Define offline_fetch [127](#) > Used in section [126](#).
 < Emit rows as CSV [122](#) > Used in section [121](#).
 < Emit rows as table [124](#) > Used in section [121](#).
 < Emit rows to DB [123](#) > Used in section [121](#).
 < Ensure a signed-in session [74](#) > Used in section [57](#).
 < Event table [15](#) > Used in section [11](#).
 < Explain the probe results [154](#) > Used in section [152](#).
 < Family and Leducky roster rows [133](#) > Used in section [132](#).
 < Fetch and print swimmer times [161](#) > Used in section [156](#).
 < Fetch events for swimmer [165](#) > Used in section [164](#).
 < Fetch events offline [166](#) > Used in section [163](#).
 < Fetch the sign-in form [68](#) > Used in section [67](#).
 < Follow the authorization callback [70](#) > Used in section [67](#).
 < Global constants [11](#) > Used in section [8](#).
 < HTML field scanner [60, 61](#) > Used in section [57](#).
 < HTTP utilities [27, 28, 30, 31, 32, 33, 34, 35, 36, 37, 39, 40, 41, 44, 48, 49](#) > Used in section [8](#).
 < Includes [10](#) > Used in section [8](#).
 < Initialize curl handle [45](#) > Used in section [44](#).
 < Insert one row [90, 91](#) > Used in section [78](#).
 < Issue HTTP request and return [47](#) > Used in section [44](#).
 < Issue person-search request [104](#) > Used in section [100](#).
 < JSON scanner [53](#) > Used in section [8](#).
 < LCM events [17](#) > Used in section [15](#).
 < Main function [132, 135, 136, 137, 146, 147, 148, 149, 151, 152, 156](#) > Used in section [8](#).
 < Match one person row [106](#) > Used in section [105](#).
 < Numeric constants [12](#) > Used in section [11](#).
 < Offline fetch [126](#) > Used in section [8](#).
 < Online swimmer dispatch [164](#) > Used in section [163](#).
 < Open DB connection [81](#) > Used in section [80](#).
 < Open and close DB connection [80](#) > Used in section [78](#).
 < Option flags [18, 21](#) > Used in section [8](#).
 < Parse BestTimes records [116](#) > Used in section [115](#).
 < Parse command-line options [157](#) > Used in section [156](#).

⟨ Perform the OIDC sign-in 66 ⟩ Used in section 57.
 ⟨ Person lookup 95, 96, 100 ⟩ Used in section 8.
 ⟨ Post the authorization code 71 ⟩ Used in section 67.
 ⟨ Print authentication advice 50 ⟩ Used in section 49.
 ⟨ Print the option keywords 140 ⟩ Used in section 139.
 ⟨ Print the sign-in note 141 ⟩ Used in section 139.
 ⟨ Print usage event codes 143 ⟩ Used in section 138.
 ⟨ Print usage examples 144 ⟩ Used in section 137.
 ⟨ Print usage flags 139 ⟩ Used in section 138.
 ⟨ Print usage options 138 ⟩ Used in section 137.
 ⟨ Print usage swimmers 142 ⟩ Used in section 138.
 ⟨ Probe data-hub endpoints 155 ⟩ Used in section 152.
 ⟨ Process one swimmer 163 ⟩ Used in section 161.
 ⟨ Read offline rows from libpq 129 ⟩ Used in section 127.
 ⟨ Read the member record 98 ⟩ Used in section 96.
 ⟨ Read the session identity 72 ⟩ Used in section 67.
 ⟨ Report missing credentials 76 ⟩ Used in section 75.
 ⟨ Report skipped swimmers 162 ⟩ Used in section 161.
 ⟨ Report the sign-in attempt 153 ⟩ Used in section 152.
 ⟨ Resolve a meet key 88 ⟩ Used in section 78.
 ⟨ Resolve a swimmer key 85, 86, 87 ⟩ Used in section 78.
 ⟨ Resolve connection string 79 ⟩ Used in section 78.
 ⟨ Return person-search result 107 ⟩ Used in section 100.
 ⟨ Run diagnostics and exit 159 ⟩ Used in section 158.
 ⟨ SCY events 16 ⟩ Used in section 15.
 ⟨ Scan person-search result 105 ⟩ Used in section 100.
 ⟨ Select cached time for event 120 ⟩ Used in section 119.
 ⟨ Sign in before fetching 160 ⟩ Used in section 158.
 ⟨ Sign in with the stored credentials 75 ⟩ Used in section 74.
 ⟨ Sign-in endpoints 14 ⟩ Used in section 13.
 ⟨ Sign-in machinery 57 ⟩ Used in section 8.
 ⟨ Sign-in transport 62, 65 ⟩ Used in section 57.
 ⟨ Skip pair if already fetched 118 ⟩ Used in section 117.
 ⟨ Sort and emit rows 121 ⟩ Used in sections 119 and 127.
 ⟨ Submit the credentials 69 ⟩ Used in section 67.
 ⟨ Synthesise a swim id 89 ⟩ Used in section 78.
 ⟨ Take the known-memberId short cut 101 ⟩ Used in section 100.
 ⟨ The swim insert statement 93 ⟩ Used in section 91.
 ⟨ Times fetch 109, 110, 112, 113, 114, 115, 117, 119 ⟩ Used in section 8.
 ⟨ Type definitions 23, 24, 25 ⟩ Used in section 8.
 ⟨ Walk the sign-in steps 67 ⟩ Used in section 66.

SWIM-TIMES

	Section	Page
Swim Times	1	1
Includes and constants	9	4
Data structures	22	8
HTTP utilities	26	9
Data-hub credentials	38	15
HTTP transport	42	17
JSON scanner	51	20
Sign-in	55	21
Database connection	77	31
Person lookup	94	37
Times fetch	108	41
Offline fetch	125	46
Main program	131	48
Glossary	167	60
Index	172	67